The Journal of Systems and Software 243 (2027) 113069

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Developers' perceptions of automated code review using a generative AI agent: The case of Gandalf☆, ☆☆

Adriano Lima Cândido [a,b,c,*], Fernando Antonio Mota Trinta [b], Lincoln Souza Rocha [b], Iury Lima Nogueira [c]

[a] Universidade Christus, Fortaleza, Ceará, Brazil
[b] Universidade Federal do Ceará, Fortaleza, Ceará, Brazil
[c] Brisanet Telecomunicações S.A., Fortaleza, Ceará, Brazil

ARTICLE INFO

Dataset link: https://doi.org/10.5281/zenodo.19866774

Keywords:
Automated code review
Generative AI
Large language models
Software Architecture
Pull requests
Empirical software engineering

ABSTRACT

Code review in pull requests (PRs) acts as a mechanism for defect detection, knowledge sharing, and renegotiation of architectural decisions. The adoption of generative agents based on LLMs in this flow advances rapidly, but an empirical gap persists regarding how developers perceive these tools in real industrial contexts. This paper reports an exploratory case study of Gandalf, a generative code-review agent that operates on GitHub PRs, deployed at Brisanet Telecomunicações S.A., a large Brazilian telecommunications company across multiple stacks. A mixed-methods study was conducted with 17 respondents, using a questionnaire based in the Technology Acceptance Model (TAM) extended to include trust in AI, and open-ended questions, which were analyzed using thematic coding, complemented by descriptive operational data from four months of the deployment. The findings indicate high self-reported acceptance (Behavioral Intention 4.37) and perceived usefulness concentrated at the PR level (Perceived Usefulness 3.92), a dominant posture of calibrated trust with human validation (8/17), and a perceived architectural impact that is indirect, limited by the absence of context beyond the diff (context-blindness in 10/17). The contributions include an industrial perception study of a diff-only LLM review agent, with a documented technical description of the agent; empirical evidence of calibrated trust; and practical requirements, articulated as design implications for future work (context, memory, and tiered analysis), for evolving from diff-level toward architecture-aware review.

1. Introduction

Code review in pull requests (PRs) is a cornerstone of modern software development and, beyond the immediate detection of defects, serves as a mechanism for knowledge sharing and for reinforcing standards of architecture, quality, and security (Badampudi et al., 2023). In industrial settings, the activity takes place under schedule pressure and over incremental changes, consumes a substantial share of engineering time, and makes PR cycle time a process metric that is sensitive to cost. Architectural decisions often surface and are renegotiated during review, which positions code review as a natural point of intervention for automated support approaches within the scope of AI for Software Architecting.

Agents based on large language models (LLMs) have been proposed as support for code review on PRs, with industrial results indicating increases in the rate of comments that lead to code changes and sustained adoption by teams (Sun et al., 2025; Cihan et al., 2025; Tantithamthavorn et al., 2026). Evidence on review time is heterogeneous: some studies report PR cycle-time reductions in specific settings (Tantithamthavorn et al., 2026), while other industrial contexts report longer PR closure times in part of the analyzed sample (Cihan et al., 2025), reinforcing the strong dependence on contextual factors. PR-oriented benchmarks expose recurring limitations, such as context sensitivity, false positives, and dependence on the technology stack (Zeng et al., 2025; Zhang et al., 2026; Gonçalves et al., 2022). Despite this technical progress, an empirical gap persists regarding how developers themselves perceive the use of generative agents in code review in real industrial settings, particularly with respect to usefulness, perceived impact on review time, trust in automated feedback,

☆ This article is part of a Special issue entitled: 'AI for SA' published in The Journal of Systems & Software.
☆☆ Editor: Prof Raffaela Mirandola.

* Corresponding author at: Universidade Federal do Ceará, Fortaleza, Ceará, Brazil.
E-mail addresses: adriano.candido@unichristus.edu.br (A.L. Cândido), fernando.trinta@dc.ufc.br (F.A.M. Trinta), lincoln@dc.ufc.br (L.S. Rocha), iury.nogueira@grupobrisanet.com.br (I.L. Nogueira).

https://doi.org/10.1016/j.jss.2026.113069
Received 8 May 2026; Received in revised form 28 July 2026; Accepted 11 August 2026
Available online 18 August 2026
0164-1212/© 2026 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

and implications for consistency and architectural decisions in the PR workflow.

This work addresses that gap through a case study of Gandalf, a generative code review agent that operates on GitHub PRs. The agent analyzes the PR diff with an LLM and publishes technical comments according to criteria that can be configured per language and domain, covering Clean Code, DRY, SOLID, and security risks. Gandalf has been deployed at Brisanet, a Brazilian telecommunications company, with multiple teams, proprietary codebases, and heterogeneous stacks (Go, Perl, Ruby, React, Angular, Next.js, Flutter/Dart). This setting combines technological diversity, delivery pressure, and multiple professional profiles, offering a concrete industrial context to examine developers' perceptions of an AI agent within the review workflow.

The objective is to empirically characterize how developers and architects perceive the adoption of a generative code review agent in a real industrial scenario, with emphasis on usefulness, reliability, and architectural implications. To this end, an exploratory mixed-methods study was conducted, anchored in a structured questionnaire grounded in the Technology Acceptance Model (TAM), extended with a trust-in-AI dimension, and complemented by open-ended questions (Section 5). The study is guided by the following research questions:

RQ1. How do developers and architects perceive the use of Gandalf in code review?
RQ2. To what extent do participants trust the feedback provided by Gandalf, and what are the perceived limitations?
RQ3. What are the perceived architectural and organizational implications of adopting Gandalf?

The focus of this work is on self-reported perceptions: objective measurements of PR cycle time, defect rates, and review coverage are out of scope and are treated as future directions. Given the single-company setting and the sample size, the study is exploratory: it aims at transferable insights rather than statistical generalization.

The main contributions of this paper are:

C1. An industrial study of developers' and architects' perceptions of a diff-only LLM code review agent in production use, grounded in TAM extended with trust in AI and paired with a documented technical description of the agent and of the structural limitations that constrain this class of agents;
C2. Empirical evidence of *calibrated trust* as the dominant reliance posture, with declared trust coexisting with systematic human validation, measured through an explicit cross-tabulation of trust and validation behavior;
C3. A set of practical requirements, elicited from participants' perceptions and from the agent's design constraints, for evolving from diff-level review toward architecture-aware review, articulated as design implications for future work within AI for Software Architecting.

The main findings indicate high self-reported acceptance, with perceived usefulness concentrated at the PR level, in particular the surfacing of issues missed by human review and the improvement of review quality. The posture of *calibrated trust* is dominant and is accompanied by systematic human validation, while the architectural impact is perceived as indirect, limited by the absence of context beyond the diff: *context-blindness* is the most reported limitation and *repo-wide-context* the most demanded improvement (Section 6).

The remainder of the paper is organized as follows. Section 2 positions the study with respect to the literature on modern code review and AI agents on PRs. Section 3 presents the studied artifact, Gandalf, including its architecture, analysis criteria, and limitations. Section 4 characterizes the organizational and technical context of the study. Section 5 details the methodological design, including participants, sampling, and data collection instrument. Section 6 reports the results, organized by the research questions. Section 7 discusses the findings in light of the literature and addresses practical implications, architectural implications for AI for Software Architecting, and threats to validity. Section 8 concludes the paper and outlines directions for future work.

2. Related work

Automated code review has gained relevance in Software Engineering, driven by the adoption of LLMs in PR-based review workflows. In practical terms, this movement seeks to reduce manual effort, improve consistency of review findings, and accelerate integration cycles, while preserving the human role in technical decision making.

In this section, the studies are organized by technical and operational dimensions, focusing on how solutions are integrated into the development workflow, what context they use for analysis, which strategies they apply to generate feedback, and how they report empirical results.

2.1. Scope and analysis dimensions

The literature considered in this section is bounded by two scoping decisions: (i) a time window from 2020 to 2026, reflecting the period in which LLM-based code review moved from prototypes to production reports; and (ii) a focus on automated code review driven by LLMs or generative AI in modern PR-based workflows, excluding purely rule-based static-analysis tools. Only studies with canonical identification through a DOI and/or an official conference, journal, or academic repository page were considered. We do not claim a systematic literature review: the goal is to position the case study against the recent state of the field, not to exhaustively map it.

The analysis dimensions adopted are: (i) integration point in the development workflow (PR, CI, or review platforms), (ii) input context used for analysis (diff-only or expanded context), (iii) technical strategy employed (rules, prompting, context retrieval, or combinations), (iv) output format and actionability for reviewers and developers, (v) reported evaluation design (metrics, user studies, and process outcomes), and (vi) limitations acknowledged by the authors.

2.2. Workflow integration and analysis context

Recent studies show convergence toward direct integration into the PR lifecycle, on platforms such as GitHub, Bitbucket, and Gerrit, with varying levels of coupling to CI pipelines (Ramesh et al., 2025; Tantithamthavorn et al., 2026; Sun et al., 2025). This convergence is consistent with the modern code review literature, in which review takes place over incremental changes under time pressure (Badampudi et al., 2023). Regarding input data, the literature points to two main strategies. The first uses only the PR diff, which tends to reduce computational cost and response time. The second incorporates additional project context, such as change history, repository information, and supporting artifacts, in order to reduce ambiguity in code interpretation (Zeng et al., 2025; Zhang et al., 2026). In practical terms, the choice between these strategies involves a trade-off between higher operational efficiency with less context and deeper analysis at a higher processing cost, with reported effects on PR cycle time that vary by configuration: cycle-time reductions in some settings and longer PR closure duration in others (Tantithamthavorn et al., 2026; Cihan et al., 2025).

2.3. Generation strategy and output actionability

There is a transition from approaches focused only on comment generation to hybrid pipelines with explicit filtering and prioritization. In industrial settings, multistage architectures seek to reduce noise and increase precision (Sun et al., 2025; Tantithamthavorn et al., 2026); in academic settings, context retrieval and dataset curation aim to increase output consistency and practical usefulness (Zhang et al., 2025; Ben Sghaier et al., 2025). As a result, the goal shifts from simply

Table 1
Comparison dimensions used to analyze the automated code review literature.

Dimension	What is analyzed
Workflow integration	Where the approach operates (PR platform, CI pipeline, or standalone review flow) and how tightly it is coupled to developer workflow.
Input context depth	Whether analysis is diff-only or enriched with repository-level context, historical data, or auxiliary artifacts.
Generation strategy	Use of rules, prompting, retrieval, multi-stage filtering, and other hybrid designs for comment generation and refinement.
Output actionability	Practical usefulness of outputs for reviewers and authors, including clarity, prioritization, and direct applicability.
Evaluation maturity	Type and strength of evidence (controlled studies, industrial-scale reports, benchmark-based evaluation, and longitudinal feedback).
Reported limitations	Recurring constraints such as false positives, domain transfer risk, context dependency, and trust-calibration requirements.

commenting on code to producing actionable feedback for technical decisions and risk prioritization.

The shift from comment generation to actionable feedback raises the bar for evaluation; this point is taken up in the next subsection, which consolidates reported practices and limitations.

2.4. Evaluation practices and reported limitations

Observed evaluation practices combine accuracy and coverage metrics with process indicators, such as PR cycle time, comment resolution rate, and developer-perceived usefulness (Cihan et al., 2025; Tantithamthavorn et al., 2026). Recent industrial studies report quantitative impact signals, including the rate of comments that lead to code changes, cycle-time reduction in specific settings, and continuous-adoption indicators (Sun et al., 2025; Tantithamthavorn et al., 2026). In parallel, PR-oriented benchmarks reinforce the need for evaluation methods closer to the real review environment, moving beyond purely textual metrics (Zeng et al., 2025; Zhang et al., 2026).

Despite this progress, recurring limitations persist consistently across study types: false positives, dependence on additional context, domain sensitivity, and the risk of trust degradation when outputs are not perceived as useful (Gonçalves et al., 2022; Cihan et al., 2025). Furthermore, process results are not uniform across projects; in some settings, perceived quality gains coexist with longer PR closure time, suggesting that effective adoption depends on fine-grained calibration of prompts, filters, and usage policy within the development workflow (Cihan et al., 2025).

2.5. Evidence profile across study types

The analyzed set presents three evidence profiles. The first comprises production studies, with stronger practical validity, which report large-scale performance and adoption behavior over time (Sun et al., 2025; Cihan et al., 2025; Tantithamthavorn et al., 2026). The second gathers benchmarks and controlled studies, with better methodological control for cross-approach comparison, but with limited generalization to heterogeneous organizational settings (Gonçalves et al., 2022; Zeng et al., 2025; Zhang et al., 2026). The third includes vision and synthesis papers, which are relevant for conceptual framing and gap identification, but should not be used in isolation as evidence of operational effectiveness (Badampudi et al., 2023; Rao et al., 2025).

This distinction by evidence profile is important for a critical reading of the literature: promising benchmark results do not translate automatically into process gains in production, and positive industrial results may depend strongly on organizational context, technology stack, and maturity of the review workflow.

Table 1 summarizes the dimensions adopted in this comparative analysis. Instead of organizing the literature by author, this structure highlights homogeneous criteria for critical reading and facilitates the identification of convergences and divergences across approaches.

Table 2 presents representative empirical signals reported in selected studies, focusing on process-related metrics and practical usefulness indicators. The goal is to make explicit which quantitative results appear most frequently in the literature and how they should be interpreted in terms of external validity and context dependence.

2.6. Levels of review feedback: Code, design, and architecture

For the purposes of this study, it is useful to distinguish three levels at which review feedback can operate. The first is *local code-quality feedback*: remarks bounded by the changed lines and their immediate surroundings, such as naming, duplication, style, localized defects, and routine security checks. The second is *design-level feedback*: remarks about module boundaries, interfaces, and patterns, which require awareness of the component or subsystem being changed, beyond the diff itself. The third is *architectural feedback* proper: remarks that engage system-wide structures, dependencies among modules, and quality-attribute trade-offs, and that presuppose repository-wide context, knowledge of the system's history, and awareness of the intent behind decisions. Code review is a natural locus where all three levels surface (Badampudi et al., 2023), but the input context available to an automated reviewer bounds the levels it can reach: diff-only agents operate, by construction, at the first level, with occasional incursions into the second when the diff happens to expose design structure. This distinction frames the reading of the present study: Gandalf is a first-level agent by design (Section 3.6), and the perceptions reported in Section 6 locate both its usefulness and its limits along these levels.

2.7. Synthesis and transition

The recent literature converges on four directions: tighter integration into the real PR workflow, gradual context expansion to reduce ambiguity, adoption of hybrid pipelines to improve feedback actionability, and broader evaluation practices beyond textual metrics. Structural limitations remain recurring, especially regarding trust in automated feedback, domain sensitivity, and performance stability across projects. The two tables complement this discussion: the first defines the analytical comparison frame; the second highlights the type of results actually observed in recent studies. This panorama provides the analytical basis for the next sections, which clarify the technical decision space in automated review and then position the studied solution based on observable criteria, reported evidence, and practical adoption constraints.

3. The Gandalf AI agent

Gandalf is an automated code review agent based on generative artificial intelligence that operates directly on GitHub PRs. The system was designed to work as a review assistant that analyzes the changes introduced in the PR (the diff) and produces technical feedback within the PR itself, using an LLM to interpret the code and generate semantic analyses grounded in software engineering principles. Rather than replacing human reviewers, the agent acts as a support mechanism. The design goals stated by the team responsible for the artifact include: automating part of the review process; flagging recurring quality issues and possible security vulnerabilities; offering improvement suggestions to developers; and contributing to uniform evaluation criteria across teams with multiple stacks. The extent to which these goals materialize in practice is an empirical question, examined in this study from the developers' perceptions (Section 6).

Table 2
Representative empirical results reported in selected automated code review studies.

Study	Key empirical signal
RovoDev (Atlassian) (Tantithamthavorn et al., 2026)	Enterprise (Bitbucket): 38.70% of generated comments triggered code changes; reported 30.8% PR cycle-time decrease in the evaluated scenario.
BitsAI-CR (Sun et al., 2025)	Industrial deployment: 75.0% precision, 26.7% outdated rate for Go, and large active-user base under a quality-control pipeline.
Automated Code Review in Practice (Cihan et al., 2025)	Multi-project industrial setting: 73.8% of automated comments resolved; PR closure duration increased in part of the sample.
SWRBench (Zeng et al., 2025)	PR-centric benchmark: underperformance of current systems and gains with multi-review aggregation, highlighting remaining quality gaps.
Sphinx (Zhang et al., 2026)	Benchmark/modeling framework: up to 40% improvement in checklist coverage in reported experimental settings.

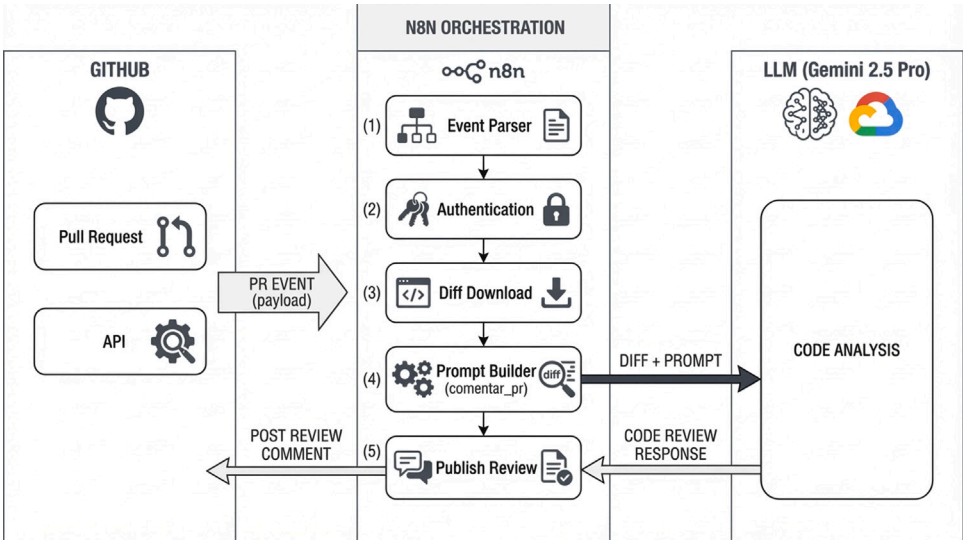


Fig. 1. Overview of the Gandalf architecture: PR event, n8n orchestration, LLM-based analysis, and posting of the review comment on the PR.

3.1. System architecture

Gandalf's architecture is based on an automation flow orchestrated by the n8n system, which acts as the workflow engine responsible for coordinating all review steps. The main components are: GitHub Webhooks; GitHub App (authentication and interaction with the API); n8n as the orchestration engine; the GitHub API; the prompt and tools module; and the Google Gemini 2.5 Pro language model. Fig. 1 summarizes the flow from the PR event to the publication of the review.

3.2. GitHub integration and authentication

Gandalf operates as a GitHub App, allowing direct interaction with PRs, comments, and repository events. Authentication uses an RSA private key (`.pem` file), JWT generation, and exchange for an installation token through the GitHub API. The flow is: (1) generate a JWT using the GitHub App private key; (2) send the JWT to the GitHub API; (3) receive the installation token; (4) use the token to access API resources (retrieve PR information, download the diff, and post comments).

3.3. Obtaining the diff and operational workflow

After authentication, the workflow retrieves the PR diff through the GitHub API. The diff represents only the changes (added and removed lines, modified files), which reduces the volume of data sent to the LLM but implies a context limitation: the agent has no visibility over the complete repository. The diff is sent to the model together with the analysis instructions.

When the review begins, Gandalf publishes a starting comment to signal that the bot is active (e.g., "Hello, adventurer, I have already started the review. Soon I will bring your scroll with the observations".); after processing, it posts a consolidated comment with suggestions, identified risks, and a final status, issued by the `comentar_pr` action (a single POST per execution, to avoid duplicates).

In addition to the automatic trigger on PR events, the agent can be invoked manually through comment commands. This possibility is necessary because Gandalf's flow is triggered only for PRs opened against the repository's default branch (`master` in the current configuration); in any other context (e.g., a PR targeting a different branch, or an already integrated branch), automatic review does not occur. This scoping also helps contain token consumption, concentrating automated review on the integration path where the team expects it to add the most value. The `/gandalf review` command forces a new full review, and `/gandalf ask [question]` allows the reviewer or developer to raise questions about Gandalf's remarks that appear incorrect or confusing, making it possible to clarify or contest the analysis.

3.4. Persona and user interaction

The agent adopts the persona of "Gandalf the White", a design choice aimed at reducing the social friction associated with receiving automated review feedback. Communication uses metaphors from The Lord of the Rings saga, empathetic language, thematic emojis, and the tone of a technical mentor. The content of the technical analysis is determined by the criteria described next, independently of the communication style.

3.5. Prompt design and analysis strategy

The analysis strategy combines technical review instructions with explicit evaluation criteria encoded in the prompt.

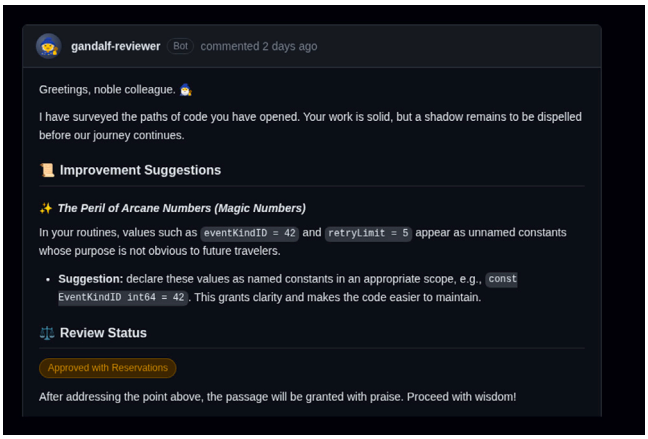


Fig. 2. Representative example of a Gandalf review comment posted on a PR (translated and sanitized for illustration).

3.5.1. Analysis criteria

The model is directed to examine: Clean Code (Martin, 2008) (clarity, readability, single responsibility of functions, method size, naming); DRY (Hunt and Thomas, 2019) (code duplication, repeated logic, opportunities for abstraction); coding style (formatting, consistency, organization); comments (unnecessary, redundant, or inconsistent); cyclomatic complexity (excessive conditionals, hard-to-follow logic, need for refactoring); SOLID principles (Martin, 2017) (single responsibility, dependency inversion, interface segregation, open-closed); and security, including recurring classes from the OWASP Top 10 (OWASP Foundation, 2021) (e.g., SQL Injection, XSS, exposure of sensitive data, authentication and authorization flaws, input validation, and use of vulnerable components), as well as error handling.

3.5.2. Technology-specific checks

The agent can be executed in any programming language and across different application contexts. The prompt is configurable and allows including language- and domain-specific checks: the same agent can be parameterized with rules targeted at Go, Dart, JavaScript, Ruby, or any other stack, as well as with criteria appropriate to the context (e.g., mobile applications, backend services, integrations). In this way, automated review can reflect the conventions, best practices, and typical risks of each ecosystem, without depending on a fixed set of technologies. In the current configuration, the prompt includes, among others, checks for Flutter/Dart (e.g., proper use of `const`, good `dispose` practices, and null safety) and for external integrations (submodule updates, impact on external APIs, and compatibility of REST and gRPC services). Performance best practices are also evaluated. Extending to new languages or contexts consists of enriching the prompt with the desired specifications, while keeping the same architecture and review flow.

3.5.3. Response format

The response follows a standardized structure. The analysis includes a list of improvement suggestions with explanations and a review status in one of three states: *Approved with Distinction* (high quality, only optional suggestions), *Approved with Reservations* (recommended improvements, but the code may be integrated), or *Not Approved* (critical issues that must be fixed before integration). Fig. 2 illustrates a representative review comment produced by the agent.

3.6. Architectural constraints of the current design

The current architectural scope privileges agility and direct integration into the PR cycle, but presents structural constraints that must be considered when interpreting the results and when designing studies about the agent.

Context limitation. The model receives only the diff and has no access to the complete repository, which may produce incomplete analyses when changes depend on broader architectural context.

Token limit. LLMs have input-size limits; PRs with more than 10,000 changed lines may cause timeouts, context truncation, or analysis failure.

False positives. The AI may flag issues in automatically generated code (e.g., from scaffolding, spec-driven generators, or low-code platforms) or in legacy code. Even with a configured prompt, it may still flag as a problem code that the organization considers adequate or accepts as technical debt: the model tends to rely primarily on generic software engineering context, without incorporating what the company adopts as standard or tolerates as technical debt.

Lack of memory. The workflow is stateless; each execution is independent and keeps no history of interactions, so the agent does not remember previous comments.

Architectural evolution directions for the agent that respond to these constraints are discussed in light of the empirical findings in Section 7.3.

4. Study context

4.1. Organizational setting

The study was conducted at Brisanet, a large Brazilian telecommunications company founded in 1998, operating in fixed and mobile telephony and related services. Brisanet serves more than two million active customers and develops its own software systems, from CRM and internal systems to customer-facing applications. This scale and the variety of software products require continuous development and maintenance, integration with telecommunications infrastructure, and consistent quality and security standards. This context is relevant for evaluating an AI-based code review agent because it combines high delivery pressure, multiple teams, and heterogeneous proprietary codebases.

4.2. Development environment and projects

Software development and operation involve product-development teams (software architects and developers) alongside DevOps, security, governance, and database functions; only the development teams are within the scope of this study (Section 5.3). On the backend, Brisanet uses Go as its main core, together with Perl and Ruby in legacy systems. On the frontend, multiple technologies coexist: React, Angular, and Next.js for web applications, and Flutter both for web and mobile. The repositories involved in the study host proprietary systems (CRM, customer-facing applications, APIs, and integrations) and use version control and code review centered on PRs. Gandalf has been gradually adopted by the teams across their repositories, so that part of the PRs receive automated feedback from the agent in addition to human review. This scenario allows for the collection of data on how developers interact with the agent and on the perceived usefulness of the tool in a real industrial context, with multiple languages and stacks.

4.3. Participants and scope

The study participants are members of technical teams that use Gandalf, including backend and frontend developers, Tech Leads/Architects, and roles adjacent to development (Scrum Master). The selection criteria and the detailed composition of the sample are described in Section 5. This section establishes the organizational and technical context so that readers can interpret the findings and assess the external validity and the generalization of the results.

5. Research method

This section describes the study design, the research questions, the selection criteria and composition of the sample, the theoretical foundation and structure of the data collection instrument, the data collection procedure, the quantitative and qualitative analysis plan, the ethical procedures adopted, and the replication package that accompanies the manuscript.

5.1. Study design

The study is conducted as an exploratory single case study with a theory-building orientation (Runeson and Höst, 2009) on the adoption of Gandalf at Brisanet (Section 4), with a mixed-methods design and a predominant quantitative component. Rather than seeking statistical generalization to a broader population, the study aims to articulate transferable insights about diff-only LLM agents that can be tested in subsequent confirmatory studies. The quantitative component operates through a structured questionnaire with five-point Likert scales, anchored in the Technology Acceptance Model (TAM) extended with trust in AI, and provides the constructs PEOU, PU, BI, Trust, and ArchImpact. The qualitative component operates through open-ended questions (Q8.1–Q8.5) submitted to thematic analysis (Section 5.6). The complementarity between the components supports triangulation of the findings around the research questions.

5.2. Research questions

The study is guided by the research questions restated from Section 1: **RQ1** asks how developers and architects perceive the use of Gandalf in code review; **RQ2** asks to what extent participants trust the feedback provided by the agent and what the perceived limitations are; **RQ3** asks the architectural and organizational implications of adoption. The correspondence between each RQ and the instruments that operationalize it is direct: RQ1 is addressed by the constructs PEOU, PU, and BI and by the open-ended question Q8.1; RQ2 is addressed by the construct Trust, by the separate validation-behavior item Q5.3, by the cross-tabulation of Q5.1 and Q5.3 (Section 5.6), and by questions Q8.2 and Q8.3; RQ3 is addressed by the construct ArchImpact and by questions Q8.4 and Q8.5.

5.3. Participants and sampling

Eligible participants are professionals who use Gandalf in the code review workflow of the repositories where the agent has been adopted (Section 4). Inclusion required interaction with the agent on at least one PR, either through automatic comments or through manual invocation. No exclusion criterion was applied based on language, stack, or project type, given the technological heterogeneity of Brisanet.

The sampling strategy was non-probabilistic by convenience. Recruitment took place through a single announcement in the internal messaging group (Google Chat) used by Brisanet's development teams; no individual invitations were sent. The announcement was posted by the first author, without intermediation by managers and without organizational pressure to respond or to evaluate the tool favorably; the stated purpose was to gather feedback to improve the tool, and participation was voluntary and unpaid. The announcement reached all 53 users of the agent who were eligible at the time, since every eligible user participates in that group; 17 chose to respond (response rate 32.1%), so the sample is self-selected. The final sample is N = 17 valid respondents, distributed among backend and frontend developers, Tech Leads/Architects, and one Scrum Master; the Scrum Master is kept in the overall sample but excluded from role-based comparisons in which $n = 1$ would render the mean meaningless. The complete sample composition (role, years of experience, prior familiarity with automated review, and exposure time to Gandalf) is reported in Section 6, Table 4, and the analytical implications of sample size are addressed in Section 5.6.

5.4. Questionnaire design and theoretical foundation

The data collection instrument is a structured questionnaire built on the TAM and on its extension with a trust dimension for AI-based systems. This foundation enables operationalizing perceptions of acceptance and use of Gandalf in a way aligned with the technology adoption literature and with the specificity of tools that rely on generative AI.

5.4.1. Adoption of the Technology Acceptance Model (TAM)

TAM, proposed by Davis (1989), is one of the most widely used models for predicting technology acceptance. It posits that two main beliefs determine intention to use and actual use: *perceived usefulness* (PU), the degree to which a person believes that using the technology improves their job performance, and *perceived ease of use* (PEOU), the belief that using the technology requires little effort. Davis (1989) developed and validated measurement scales for PU and PEOU (six-item scales each, with high reliability), widely replicated and adapted in diverse contexts. TAM was chosen because it offers a simple, well-validated framework suitable for capturing perceived usefulness, ease of use, and behavioral intention to use in relation to a specific tool, in this case Gandalf.

5.4.2. Extension with trust in AI

In AI contexts, trust in the technology is a critical factor for acceptance, often as relevant as usefulness and ease of use (Gefen et al., 2003). The integration of trust with TAM has been used in settings where the adoption decision depends on the credibility and predictability of the system (e.g., e-commerce, automated systems). For generative AI tools, user trust in the system's recommendations and suggestions is especially important, since the output is not deterministic and the developer must decide to what extent to accept or validate the suggestions. The extension is supported by works that combine TAM and trust in AI to explain adoption (Alshammari and Al-Mamary, 2025) and by instruments that evaluate trust and acceptance of AI technologies in professional settings (Stevens and Stetson, 2023).

5.4.3. Instrument structure

The questionnaire is composed of eight blocks, mapped to the theoretical constructs and to the items used in the analysis. Table 3 summarizes this correspondence. The full instrument, translated into English, is reproduced in Appendix.

The 19 items in blocks 3–7 were adapted to the context of the code review agent (e.g., learning within the code review flow, CI/CD integration, impact on quality and on the speed of the review cycle), keeping the five-point structure. Item q5_3 ("I felt the need to manually validate the suggestions") is worded in the reverse direction of the declared-trust items. Because it captures validation behavior rather than declared trust, it is not aggregated into the Trust construct, which comprises q5_1 and q5_2, and is analyzed as a separate single-item measure, kept on the original scale for the cross-tabulation Q5.1 × Q5.3 (Section 5.6). Block 8 contains five open-ended questions that cover perceived benefits (Q8.1), observed limitations (Q8.2), low-trust scenarios (Q8.3), required improvements (Q8.4), and willingness to use the agent on critical projects (Q8.5).

5.5. Data collection procedure

The questionnaire was made available in Google Forms in online mode and answered in self-administered form by the participants, in Portuguese, between April 1 and April 21, 2026, with an estimated average completion time of approximately 15 min. The first question of the questionnaire records informed consent (Section 5.7); only responses with explicit consent were included in the final sample, and the 17 valid responses reported in this study correspond entirely to participants who consented. The questionnaire did not require direct identification of respondents.

Table 3
Structure of the data collection instrument. Blocks 3–7 use a five-point Likert scale (from "Strongly disagree" to "Strongly agree"). Block 8 contains open-ended questions analyzed thematically.

Block	Function	Construct	Items	Role in the study
1	Participant profile	n/a	q1_1–q1_4	Sample characterization (role, experience, prior familiarity, exposure time)
2	Project context	n/a	q2_2–q2_6	Scenario characterization (q2_3 = system_domain; q2_1 discarded at load time, see Section 5.7)
3	Perceived ease of use	PEOU	q3_1–q3_4	TAM independent variable
4	Perceived usefulness	PU	q4_1–q4_5	TAM independent variable
5	Trust in AI	Trust	q5_1–q5_3	TAM extension (trust: q5_1, q5_2); q5_3 is a separate validation-behavior measure
6	Behavioral intention	BI	q6_1–q6_3	TAM dependent variable
7	Architectural and organizational impact	ArchImpact	q7_1–q7_4	Impact variable (linked to RQ3)
8	Open-ended questions	n/a	q8_1–q8_5	Qualitative component (thematic analysis)

5.6. Data analysis

The analysis was conducted in a reproducible Python pipeline, described in Section 5.8, and produces the tables and figures reported in this manuscript (Tables 4, 5, 7, 6, 8, and 9; Figs. 3, 4, and 5), as well as supplementary subgroup figures included only in the replication package.

5.6.1. Quantitative analysis

Likert responses were deterministically coded to the 1–5 scale ("Strongly disagree" = 1; . . . ; "Strongly agree" = 5). For each construct, item-level descriptive statistics (median, mean, standard deviation, and percentage of agreement at 4–5) and aggregate statistics per construct (mean and Cronbach's alpha) were computed. Internal consistency was estimated using the standard Cronbach's alpha formula on the items-by-respondents matrix, complemented by the mean inter-item correlation and the Spearman–Brown coefficient; for the two-item Trust construct (q5_1, q5_2), alpha is unstable and the inter-item correlation with Spearman–Brown is the preferred reading. Item q5_3 does not enter any construct aggregate and is described as a separate validation-behavior measure (Section 5.4). The coefficients reported in Section 6 (Table 6) follow this procedure.

The operationalization of the *trust* × *validation* quadrant (Table 8) crosses Q5.1 (declared trust) and Q5.3 (need for manual validation). Each axis is dichotomized into "agrees" (response 4 or 5) and "other" (1 to 3), producing four interpretable quadrants: *calibrated trust* (high × high), *blind trust* (high × low), *defensive distrust* (low × high), and *disengagement* (low × low).

Owing to the sample size (N = 17), the analysis is restricted to descriptive statistics, without inferential tests; subgroup breakdowns (by role, exposure time) reported in Section 6 are presented for illustrative purposes and do not support formal inference.

5.6.2. Qualitative analysis

The responses to the five open-ended questions (Q8.1–Q8.5) were submitted to thematic analysis with a code book defined a priori and iteratively refined during coding. Coding is multilabel (a single response can receive multiple codes) and was initially conducted by the first author. All 85 cells (17 respondents × 5 questions) were then independently re-coded, blind to the original assignments and working from the code book alone, by two additional coders: an external researcher with no ties to the project or the company, and a co-author with no ties to Brisanet who did not participate in the original coding. Over the binary code-by-cell decisions, raw agreement with the original coding was 0.893 and 0.861 (pooled Cohen's κ of 0.705 and 0.599), and Krippendorff's alpha across the three coders was 0.635, reported as a robustness check because several codes are rare (assigned to only a few respondents), a regime in which κ is unstable; per-code agreement is included in the replication package. Divergences were resolved by a two-of-three majority vote followed by discussion of the cells flagged by the coders; in three cells the discussion retained the original assignment on explicit textual grounds, and the cell-by-cell provenance is recorded in the replication package. The final assignments and the count per code are reported in Table 9 and in Fig. 5. The respondent quotes used in the text were translated from Portuguese into English following a translation rule defined a priori, with the original version preserved in the coding files of the replication package.

5.7. Ethics

Participation was voluntary and preceded by informed consent recorded as the first question of the questionnaire; only responses with explicit consent (`Yes`) are part of the sample (all 17 valid responses). Anonymization was handled in a preventive way at data load time: question Q2.1, which asked for the name of the participant's system, was discarded at load and is not part of any analysis. Question Q2.3 (project purpose) was normalized to a controlled English vocabulary (`system_domain`) before any aggregation, avoiding free-text descriptions that could expose organizationally sensitive information. No data of direct identification of respondents is presented in the manuscript or in the artifacts of the replication package. Additional mitigations regarding potential conflict of interest (the Gandalf agent is developed at Brisanet, the same organization in which the study was conducted) are discussed in Section 7.5.

5.8. Replication package

A public replication package accompanies the manuscript and is archived on Zenodo at https://doi.org/10.5281/zenodo.19866774. The package is organized in `scripts/` (`load_data.py`, `code_likert.py`, `code_q8.py`, `run_analysis.py`, `plot_style.py`), `tables/` (T1–T7 in CSV and LaTeX, plus `q8_codes.csv` with the thematic codes per respondent and the English translation used in the manuscript), `figures/` (F1–F7 in PDF and PNG), `codebook/` (data dictionary and Q8 code book), and a curatorial summary in `REPORT.md`. T1–T7 and F1–F7 are the artifact names inside the archive: T1–T6 correspond to Tables 4, 5, 7, 6, 8, and 9 of this manuscript, T7 corresponds to Table 10, and F1, F2, and F7 correspond to Figs. 3, 4, and 5, respectively; F3–F6 are supplementary subgroup views not included in the manuscript. The aggregated artifacts made available reproduce all numbers reported in the manuscript without requiring access to the individual raw responses, which are not redistributed in order to preserve participant confidentiality; in particular, the original texts of the open-ended responses are intentionally omitted from the public package. Likewise, the deployment aggregates of Section 6.5 enter the package as the script `deployment_metrics.py` and the artifact T7 only; the raw review logs are not redistributed, since they contain repository and user identifiers. The inter-rater reliability artifacts of Section 5.6.2 are included under `irr/`, together with the script `irr_q8.py`: the blind coding sheets with the response texts removed, the agreement statistics, and the cell-by-cell resolution provenance. Full reproduction of the pipeline from raw responses, when locally available, is performed by a single command (`python scripts/run_analysis.py`).

Table 4
Demographic profile of the participants (N = 17).

Category	Level	n	pct
Role	Developer	12	70.6%
	Tech Lead/Architect	4	23.5%
	Other	1	5.9%
Experience	2–5 yrs	11	64.7%
	6–10 yrs	6	35.3%
Prior automated review familiarity	None	2	11.8%
	Low	4	23.5%
	Moderate	10	58.8%
	High	1	5.9%
Exposure to Gandalf	1–3 mo	2	11.8%
	3–6 mo	4	23.5%
	>6 mo	11	64.7%

Table 5
Project and system context reported by the participants (N = 17).

Category	Level	n	pct
Involvement (Q2.2)	Low	3	17.6%
	Medium	3	17.6%
	High	11	64.7%
Organizational impact (Q2.4)	Medium	1	5.9%
	High	16	94.1%
Lifecycle stage (Q2.5)	Early development	1	5.9%
	Continuous evolution	9	52.9%
	Stable	4	23.5%
	Legacy (mostly maintenance)	3	17.6%
Team size (Q2.6)	1–5 people	9	52.9%
	6–10 people	6	35.3%
	>20 people	2	11.8%
System domain (Q2.3, controlled)	Billing	2	11.8%
	CRM	2	11.8%
	Customer service	8	47.1%
	Data engineering	1	5.9%
	Device provisioning	2	11.8%
	Internal innovation	1	5.9%
	Sales	1	5.9%

6. Results

This section reports the empirical findings organized by the research questions formulated in Section 5: Section 6.1 characterizes the sample; Section 6.2 answers RQ1, on perceived use (PEOU, PU, and BI); Section 6.3 answers RQ2, on trust and limitations; and Section 6.4 answers RQ3, on architectural and organizational implications. Section 6.5 complements the perception results with descriptive operational data from the deployment. In line with the analytical decisions detailed in Section 5.6, the presentation is descriptive: means, medians, agreement percentages, Cronbach's alpha per construct, and thematic synthesis of the open-ended questions. The qualitative quotes are identified by the respondent ($R\langle id \rangle$) and by the system domain (*system_domain*, controlled vocabulary).

6.1. Sample

Of the 17 respondents who consented to participate and completed the instrument, the majority (12, 70.6%) work as developers, with 4 tech leads/architects and 1 Scrum master. Professional experience is concentrated in the intermediate range: 11 respondents (64.7%) reported between 2 and 5 years of career and 6 (35.3%) between 6 and 10 years.

Regarding prior familiarity with automated code review tools, the distribution is mostly moderate (10, 58.8%) or low (4, 23.5%); only 3 respondents are at the extremes (2 with no prior familiarity, 1 with high familiarity). Exposure to Gandalf, in turn, concentrates on prolonged use: 11 respondents (64.7%) reported more than 6 months of use, 4 (23.5%) between 3 and 6 months, and 2 (11.8%) between 1 and 3 months. Table 4 summarizes the four demographic dimensions.

Table 5 characterizes the context of the projects and systems involved. Regarding current involvement with the system (Q2.2), measured as the frequency and centrality of activity in the project, 11 respondents (64.7%) reported a high level (frequent or central activity), and the remaining 6 are split equally between medium (recurring participation) and low (sporadic interaction). The declared organizational impact (Q2.4) is practically uniform: 16/17 (94.1%) reported high impact, evidencing that the study captures predominantly business-relevant systems. The life-cycle stage concentrates on continuous evolution (9, 52.9%) and stable (4, 23.5%), with 3 respondents in legacy/maintenance and 1 in early development.

Predominant team size is small to medium: 9 respondents in teams of 1–5 people, 6 in teams of 6–10, and 2 in teams above 20. The system domain, normalized from Q2.3 to a controlled English vocabulary, is dominated by customer service (8, 47.1%), followed by billing, CRM, and device provisioning (2 each), and by data engineering, internal innovation, and sales (1 each). The 17 respondents thus come predominantly from small teams that maintain systems in continuous evolution, with a strong focus on customer service, and exhibit prolonged use and moderate familiarity with automated review, conditions that delimit the external validity of the findings reported next.

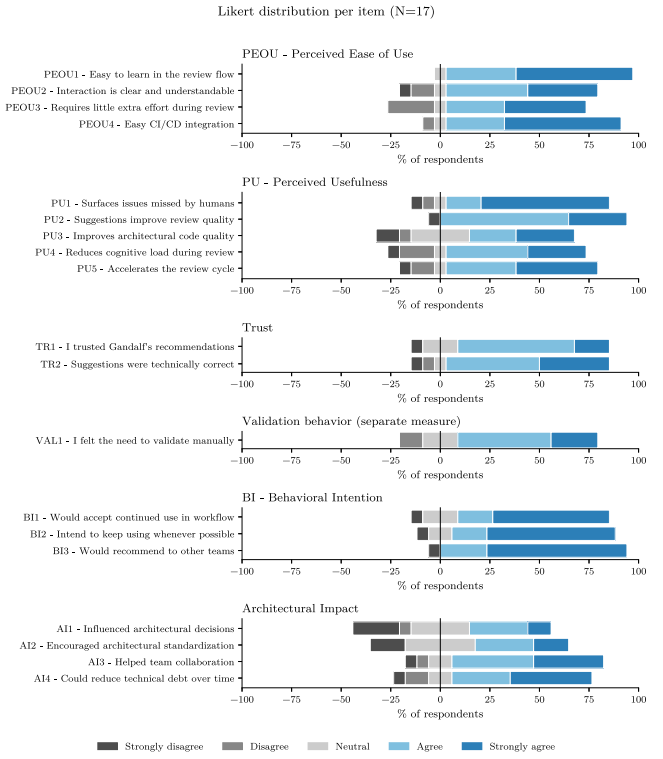


Fig. 3. Distribution of agreement across the 19 Likert items, grouped by construct (PEOU, PU, Trust, BI, ArchImpact); the validation-behavior item (q5_3), analyzed separately from Trust, is displayed in its own panel. Negative responses (1–2) extend leftward, positive responses (4–5) rightward, and neutral responses (3) are split across the center.

6.2. RQ1: Perceived ease of use, usefulness and behavioral intention

Fig. 3 presents the agreement distribution of the 19 Likert items grouped by construct, and Table 6 reports the internal consistency estimated via Cronbach's alpha. Table 7 contains the item-by-item statistics (median, mean, standard deviation, and agreement percentage) cited specifically in this section.

At the item level, *Perceived Ease of Use* (PEOU) shows agreement along two fronts. Learning the tool within the review flow (q3_1)

Table 6
Internal consistency per construct (N = 17): Cronbach's alpha, mean inter-item Pearson correlation (r), and Spearman–Brown coefficient (S-B). Trust comprises the two declared-trust items (Q5.1, Q5.2); Q5.3 is analyzed as a separate validation-behavior measure. For two-item scales, alpha is unstable and the inter-item correlation with Spearman–Brown is the preferred reading.

Construct	k	Alpha	r	S-B
PEOU	4	0.586	0.279	0.608
PU	5	0.921	0.709	0.924
Trust	2	0.860	0.764	0.866
BI	3	0.963	0.901	0.965
ArchImpact	4	0.876	0.644	0.879

reached a mean of 4.53 and 94.1% agreement, and ease of CI/CD integration (q3_4) stood at 4.41 and 88.2%.

The two central items of the construct, related to clarity of interaction (q3_2) and additional effort during review (q3_3), presented lower means (3.88 in both) and higher dispersion (standard deviation 1.22), with 76.5% and 70.6% agreement, respectively. Internal consistency is low (α = 0.586, below the conventional threshold of 0.7), and the four items cover partially distinct subdimensions (learning, clarity, effort, CI/CD integration); the PEOU reading in this study is therefore anchored in the individual items, and the aggregate mean (4.18) is reported as a description only, with no conclusion resting on it.

The aggregate *Perceived Usefulness* (PU) mean was 3.92 (α = 0.921). Agreement is highest in the items that touch the immediate effect on review: detection of issues that escape human review (q4_1, mean 4.29; 82.4% agreement) and improvement in review quality (q4_2, mean 4.12; 94.1% agreement). The items that connect the agent to more diffuse effects exhibit lower means: cycle acceleration (q4_5, 3.94), cognitive load reduction (q4_4, 3.71), and architectural code quality (q4_3, 3.53; 52.9% agreement, the lowest within the construct). The contrast between q4_1/q4_2 and q4_3 establishes a recurring pattern across the other constructs: perceived usefulness concentrated at the diff level and more discreet at the architectural level.

The aggregate *Behavioral Intention* (BI) mean was 4.37 (α = 0.963, the highest among the five constructs). The three items converge on high agreement: continuing to use the tool in the workflow (q6_1, mean 4.24; 76.5% agreement), intending to keep using it whenever possible (q6_2, mean 4.35; 82.4%), and recommending it to other teams (q6_3, mean 4.53; 94.1%). Across PEOU, PU, and BI, the observed sample perceives Gandalf as easy to integrate and useful at the PR level, and declares the intention of continued use.

The thematic analysis of the open-ended responses to Q8.1 (perceived benefits) reinforces this picture. The most frequent code was *unseen-issue-detection*, assigned to 11/17 (64.7%), followed by *code-standardization* (4), *cycle-acceleration* and *team-collaboration-support* (3 each). A respondent from the customer service area (R12) summarizes the dominant pattern: *"The machine never gets tired of reading code, so it doesn't let small errors and inconsistencies slip through, things that human analyses miss due to fatigue, especially on large PRs"*. The standardization strand appears in another respondent from the same domain (R13): *"The main perceived benefits were identifying off-standard names, suggesting improvements to make them clearer and aligned with project conventions, and detecting repeated code snippets, encouraging reuse and cleaner organization"*. The reported benefits, therefore, concentrate on dimensions local to the PR (specific issues, standardization, and speed), in line with the high agreement observed in q4_1, q4_2, and q3_1.

6.3. RQ2: Trust and perceived limitations

The *Trust* construct gathers the two declared-trust items (q5_1, q5_2), whose distribution is presented in the *Trust* panel of Fig. 3; the validation-behavior item (q5_3) is analyzed separately and displayed in the adjacent panel. The aggregate Trust mean was 3.91, with an inter-item correlation of r = 0.764 and a Spearman–Brown coefficient of 0.866 (Table 6; with two items, Cronbach's alpha, 0.860, is reported for completeness). The item-by-item pattern (Table 7) shows that respondents consider the recommendations technically correct (q5_2, mean 4.00; 82.4% agreement) and trust them in general (q5_1, mean 3.82; 76.5% agreement). The separate validation-behavior measure shows that, at the same time, respondents report a high need to manually validate the suggestions (q5_3, mean 3.82; 70.6% agreement): declared trust coexists with sustained manual validation, and few respondents would dispense with the latter.

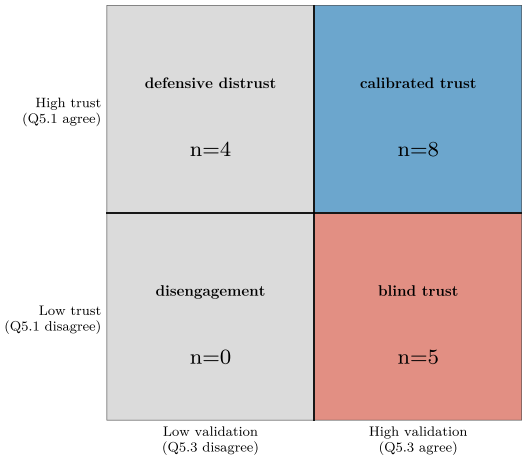


Fig. 4. Joint pattern of declared trust (Q5.1) and need for manual validation (Q5.3), dichotomized at agreement (scores 4–5); cell counts denote respondents per quadrant (N = 17).

Applying the operationalization of the *trust* × *validation* quadrant described in Section 5.6, Fig. 4 illustrates the result and Table 8 reports the counts. The predominant quadrant is *calibrated trust* (Q5.1 high and Q5.3 high), with 8/17, followed by *blind trust* with 5, *defensive distrust* with 4, and *disengagement* with none. The combination of declared trust with the maintenance of manual validation thus concentrates almost half of the sample.

The open-ended responses to Q8.2 (perceived limitations) and Q8.3 (distrust scenarios) make this conservative posture explicit. In Q8.2, the most frequent code was *context-blindness*, assigned to 10/17 (58.8%), with *business-rules-blindness* in 4 and *architectural-blindness* in 3. A respondent from the customer service area (R15) illustrates the lack of contextual view in long-evolving systems: *"On small fixes (since the customer service system, despite being over four years old, has never reached a stable moment because of constant improvements and new features), there's a lot of intertwined code, and on small fixes the tool brings suggestions that don't stay confined to what was changed but extend to the file itself"*. Another respondent from the same domain (R4) generalizes the criticism to multiple dimensions of context not accessible to the agent: *"It does not know business rules, project architecture, the task context behind the PR, the project-wide 'utils' functions, the differences between layers; error handling is problematic and contextless [...] suggestions are often nonsensical [...]"*. The criticism extends to false positives, as reported by a respondent from the device provisioning area (R10): *"Sometimes it labels something as 'bad' that, in the feature's context, is actually necessary or not really bad as presented"*; and to the absence of memory, as noted by a respondent from the customer service area (R9): *"It doesn't re-evaluate after you fix what it requested"*.

In Q8.3, the scenarios in which participants explicitly report they would distrust the agent are led by *complex-business-logic* (7 respondents, 41.2%), followed by *business-critical* and *security-sensitive* (2 each) and *architectural-decisions* (1). A respondent from the data engineering area (R8) describes the type of review in which they would reduce trust in the agent: *"Code reviews that depend on a broader task context. For example, modifying parts of the code that communicate with an old*

Table 7
Item-level descriptive statistics for the 19 Likert items (N = 17). Pct. agree is the share of responses scoring 4 or 5. Item q5_3 (VAL1) is analyzed as a separate validation-behavior measure, not as part of the Trust construct.

Item	Label	Median	Mean	sd	pct_agree
q3_1	PEOU1 — Easy to learn in the review flow	5.0	4.53	0.62	94.1%
q3_2	PEOU2 — Interaction is clear and understandable	4.0	3.88	1.22	76.5%
q3_3	PEOU3 — Requires little extra effort during review	4.0	3.88	1.22	70.6%
q3_4	PEOU4 — Easy CI/CD integration	5.0	4.41	0.87	88.2%
q4_1	PU1 — Surfaces issues missed by humans	5.0	4.29	1.21	82.4%
q4_2	PU2 — Suggestions improve review quality	4.0	4.12	0.93	94.1%
q4_3	PU3 — Improves architectural code quality	4.0	3.53	1.33	52.9%
q4_4	PU4 — Reduces cognitive load during review	4.0	3.71	1.26	70.6%
q4_5	PU5 — Accelerates the review cycle	4.0	3.94	1.25	76.5%
q5_1	TR1 — I trusted Gandalf's recommendations	4.0	3.82	0.95	76.5%
q5_2	TR2 — Suggestions were technically correct	4.0	4.00	1.12	82.4%
q5_3	VAL1 - I felt the need to validate manually	4.0	3.82	0.95	70.6%
q6_1	BI1 — Would accept continued use in workflow	5.0	4.24	1.15	76.5%
q6_2	BI2 — Intend to keep using whenever possible	5.0	4.35	1.11	82.4%
q6_3	BI3 — Would recommend to other teams	5.0	4.53	1.01	94.1%
q7_1	AI1 — Influenced architectural decisions	3.0	3.00	1.37	41.2%
q7_2	AI2 — Encouraged architectural standardization	3.0	3.29	1.31	47.1%
q7_3	AI3 — Helped team collaboration	4.0	3.94	1.14	76.5%
q7_4	AI4 — Could reduce technical debt over time	4.0	3.88	1.27	70.6%

Table 8
Joint distribution of declared trust (Q5.1) and the separate validation-behavior item (Q5.3, need for manual validation), dichotomized at agreement (scores 4–5).

trust_q5_1	validation_q5_3	Label	n
High (4–5)	High (4–5)	Calibrated trust	8
High (4–5)	Low (1–3)	Blind trust	5
Low (1–3)	High (4–5)	Defensive distrust	4
Low (1–3)	Low (1–3)	Disengagement	0

project or with larger structures that aren't ready yet and it flags them as wrong". One respondent departs from the majority pattern altogether (R4, customer service): *"I never trust Gandalf's review"*. The quantitative quadrant data and the qualitative evidence converge on the same posture quantified in the quadrant analysis, and the distrust zones concentrate on situations that depend on context not captured by the diff.

6.4. RQ3: Architectural and organizational implications

The *Architectural Impact* (ArchImpact) construct is presented in the corresponding panel of Fig. 3, with aggregate mean 3.53 and Cronbach's alpha $\alpha = 0.876$ (Table 6). The item-by-item profile (Table 7) is heterogeneous: agreement is highest in the item related to support for team collaboration (q7_3, mean 3.94; 76.5% agreement) and reasonable as for the reduction of technical debt over time (q7_4, mean 3.88; 70.6%).

The items central to the architectural dimension, however, show more discreet agreement. The encouragement of architectural standardization (q7_2, mean 3.29) reaches 47.1% agreement, and the declared influence on architectural decisions (q7_1, mean 3.00) reaches only 41.2%, with median 3 and standard deviation 1.37. These two items are, among the 19 Likert items, those with the lowest aggregate agreement: Gandalf is more recognized for reinforcing consistency and collaboration at the PR level than for affecting architectural choices proper.

The thematic analysis of the open-ended responses to Q8.4 (required improvements) and Q8.5 (use on critical projects) elaborates this reading. In Q8.4, the most frequent code was *repo-wide-context*, assigned to 10/17 (58.8%), followed by *language-domain-config* (6, 35.3%), *human-feedback-loop* (3), and *learning-from-history* (2); the code *severity-tiers* was not assigned to any respondent. A respondent from the customer service area (R4) summarizes the recurring request for context beyond the diff: *"Per-project specialization. Gandalf doesn't have enough context for code review [...] It must 'know' what each layer of the project does, 'know' the 'utils' and 'oops' packages, have basic 'understanding' of business rules and access to the PR description and the Jira task to 'understand' what it is analyzing"*.

Another respondent from the same domain (R3) adds product and temporal feedback suggestions: *"Have a simpler-to-pronounce name or a different command to invoke it in pull requests ('/review', '/ask'). I'd remove the fluff about the character behind the name, since it makes the recommendations tedious to read; keep it objective and technical. I'd also like a weekly statistical and descriptive report on the team's PRs to understand the most cited problems and better guide the team"*.

In Q8.5, on use in critical projects, the majority codes were *would-use-as-first-pass* (12 respondents, 70.6%) and *would-use-with-double-review* (4, 23.5%), with *conditional-on-improvements* in 2 and only 1 respondent in *would-not-use*. A respondent from the customer service area (R12) describes the form perceived as safe for use: *"Yes, but not without a human reviewer supervising the same PRs. In my view Gandalf serves to catch errors humans missed, mostly at the file or at most task level. Large and complex projects still need humans who understand the rules, concepts, and decisions"*. The single negative voice is explicit about its reasons (R4, customer service): *"No; currently, Gandalf looks more like a poorly configured linter than a code review agent"*.

Fig. 5 consolidates the frequencies of the thematic codes per question Q8.1–Q8.5, and Table 9 exemplifies the codes with representative quotes.

6.5. Deployment context: Descriptive operational data

To situate the perceptions reported above in the operational reality of the deployment, this subsection reports descriptive aggregates of Gandalf's activity between January and April 2026, a window that contains the survey period (Section 5.5). This window also corresponds to the configuration studied in this paper: the agent's prompt was iteratively adjusted from developer feedback during a limited 2025 pilot restricted to strategic repositories, and the deployment was expanded to the broader set of the company's projects in 2026, so the window captures the stabilized configuration under broad coverage. The source is the complete set of review logs posted by the agent on GitHub, one log per PR, extracted by the company from the repositories in which the agent was active. Because the agent does not run on every PR of the organization, all denominators refer to PRs in which

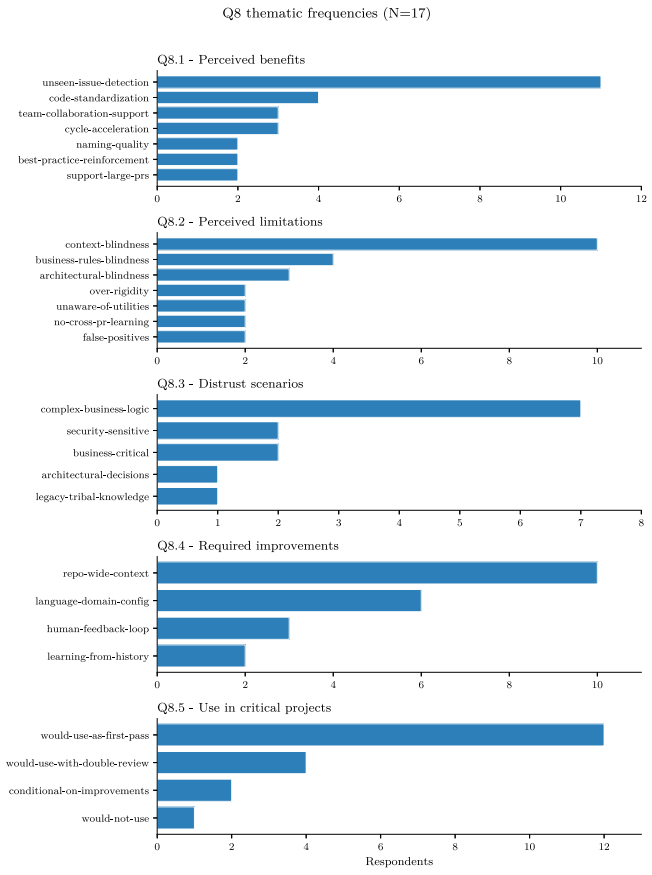


Fig. 5. Frequency of thematic codes for the five open-ended questions Q8.1–Q8.5 (N = 17); codes within each panel are ordered by frequency.

Table 9
Frequency of thematic codes assigned to the open-ended responses Q8.1–Q8.5 (N = 17).

Question	Code	Frequency
q8_1	unseen-issue-detection	11
q8_1	code-standardization	4
q8_1	support-large-prs	2
q8_1	cycle-acceleration	3
q8_1	team-collaboration-support	3
q8_1	best-practice-reinforcement	2
q8_1	naming-quality	2
q8_2	context-blindness	10
q8_2	false-positives	2
q8_2	no-cross-pr-learning	2
q8_2	unaware-of-utilities	2
q8_2	over-rigidity	2
q8_2	business-rules-blindness	4
q8_2	architectural-blindness	3
q8_3	business-critical	2
q8_3	complex-business-logic	7
q8_3	legacy-tribal-knowledge	1
q8_3	security-sensitive	2
q8_3	architectural-decisions	1
q8_4	repo-wide-context	10
q8_4	learning-from-history	2
q8_4	severity-tiers	0
q8_4	language-domain-config	6
q8_4	human-feedback-loop	3
q8_5	would-use-with-double-review	4
q8_5	would-not-use	1
q8_5	would-use-as-first-pass	12
q8_5	conditional-on-improvements	2

Table 10
Descriptive operational data of the Gandalf deployment, January–April 2026.

Metric	Value
Coverage	
PRs reviewed (Gandalf invoked)	603
Repositories covered	38
PRs per month (Jan/Feb/Mar/Apr)	121/131/139/212
PRs excluded (no complete review)	24
PRs analyzed (at least one verdict)	579
Review cycles parsed/without a recognizable verdict	822/39
Verdicts (review events)	
Total verdict events	783
Approved (no reservations)	147
Approved with reservations	360
Not approved	276
Final verdict per PR	
Approved (no reservations)	95
Approved with reservations	302
Not approved	182
Re-review cycle	
PRs with more than one review cycle	100 (17.3%)
Verdict improved after re-review	41
Verdict regressed after re-review	12
Verdict unchanged after re-review	47
Invocation mode	
Automatic (PR opened against the default branch)	418 (69.3%)
Manual (developer command)	185 (30.7%)

Gandalf was invoked, automatically or manually, and not to all PRs of the period. The aggregates in Table 10 are recomputed from the raw logs by a reproducible script distributed in the replication package (Section 5.8), except for the two invocation-mode counts, which were extracted by the company from the GitHub events; repository identities are anonymized, and no repository, system, or user identifier is retained in any published artifact. Exclusions are explicit: in 24 of the 603 PRs an invocation failure prevented the agent from posting a complete review, and 39 of the 822 review cycles carry no recognizable verdict; both sets are counted in Table 10 and excluded from the verdict aggregates.

Gandalf was invoked on 603 PRs across 38 repositories, with monthly volume growing from 121 PRs in January to 212 in April. The 579 PRs with at least one complete review received 783 review verdicts: 147 approved without reservations (the *Approved with Distinction* state of Section 3), 360 approved with reservations, and 276 not approved. According to the classification produced by the company for the same window, the reviews raised 1932 individual findings, each categorized observation within a review comment (about 3.2 per reviewed PR), most frequently general improvement suggestions and contextual observations, followed by code duplication, naming conventions, and SQL injection risks; we report this classification as provided by the company rather than recomputing it. The invocation mode follows the operational rule of Section 3.3: 418 invocations (69.3%) were triggered automatically by the opening of a PR against the repository's default branch, and 185 (30.7%) were explicit developer commands (`/gandalf review` or `/gandalf ask`); these two counts were extracted by the company from the GitHub events.

The re-review cycle is the one point of contact between the operational data and the survey's perception measures. In the recomputed aggregates, 100 PRs (17.3% of those with a complete review) went through more than one review cycle; among these, the final verdict improved in 41 PRs, regressed in 12, and remained unchanged in 47. The 41 improvements correspond to PRs in which the developer adjusted the code after the agent's feedback until the verdict changed for the better, a behavioral trace, rather than a self-reported perception, that the agent's feedback prompts action. This trace does not establish that the resulting changes improved the software, and the logs contain no link between individual survey respondents and PRs; these aggregates

Table 11
One sanitized example per category of Gandalf review comment, drawn from the January–April 2026 deployment logs (translated from Portuguese; system, file, and business identifiers removed). Verdicts refer to the quoted review cycle.

Category	Excerpt of the agent's comment	Context and outcome
Useful	"Your intention of blocking evil with a custom `SqlInjectionVerify` function is noble. However, the truly powerful magic against SQL injection lies in prepared statements and placeholders. This ancient technique separates the command from the data, making injection impossible. Suggestion: remove the custom check and pass the user-supplied values to the query exclusively through placeholders".	First cycle: not approved. In the next cycle the agent confirms that the concatenation was replaced by a parameterized query and approves the PR: the developer acted on the finding. Complements the representative example of Fig. 2.
False positive	"A time-calculation function was removed from the database schema, and its *old* variant was renamed to take its place. This is a powerful spell that affects the heart of the system. It is crucial to be certain that the old logic is indeed the one that should prevail. Confirm that this change is intentional".	The PR is explicitly a revert whose title states that it restores the exact state of a previous commit; the agent questions the declared purpose of the PR as potentially unintentional. Approved with reservations.
Context-blind	"A crucial `JOIN` was removed from the main query. Previously, this join ensured that only tickets from cities with a given configuration flag enabled were selected; now all tickets matching the other criteria will be selected regardless of that configuration. Was this your intention? Has the filtering logic been moved elsewhere, or is this business rule no longer necessary?"	The PR title states the business motivation: the validation no longer applies after a migration. Seeing only the diff, the agent treats the intentional removal of the rule as a possible defect and repeats the question in both review cycles. Approved with reservations.
Potentially architecture-relevant	"The main change here is the update of a service submodule. Although the change in the main repository is small, the consequences can be vast. Please detail in the PR description the main changes contained in this update. Were any contracts broken? If communication interfaces (gRPC or REST contracts) were changed, are the consuming services prepared for that?"	Single cycle: not approved until the impact of the submodule update is described. The agent demands reasoning about contract compatibility across services that goes beyond the one-line diff of the submodule hash.

therefore complement, but do not validate, the perceptions reported in Sections 6.2 to 6.4.

The deployment offers no mechanism for PR authors or human reviewers to flag an incorrect suggestion, so a false-positive rate cannot be objectively recovered from these logs: a suggestion that was not applied may equally be a valid suggestion that the team deprioritized, a defensible style disagreement, or part of an abandoned PR, and single-cycle PRs do not reveal whether the developer applied anything at all. Suggestions that persist across re-review cycles would provide, at best, a noisy upper bound. The proportion of comments accepted or resolved and the review cycle time before and after adoption are also not recoverable: the logs contain only the agent's own comments, and the n8n orchestration that runs the agent is stateless, retaining no per-comment resolution state and no PR timeline. This observed gap directly motivates the flagging safeguard discussed in Section 7.4: instrumenting the deployment with such a mechanism would make the false-positive rate measurable, and we leave that instrumentation, together with the collection of acceptance and cycle-time metrics, as future work; as a first step, the generation of logs covering the full review cycle is under development as an improvement to the tool.

To make the quality of the agent's output concrete, Table 11 presents one sanitized example per category of review comment observed in the logs: useful, false positive (a comment that conflicts with the stated purpose of the PR, the criterion used in the absence of a measurable false-positive rate), context-blind, and potentially architecture-relevant. These are the same categories that organize the perceived limitations reported in Sections 6.3 and 6.4, and the *useful* category is anchored by the representative comment already reproduced in Fig. 2. The excerpts were translated from Portuguese and sanitized: system, file, and business identifiers were removed and the texts shortened, preserving the technical content and traces of the agent's persona. Because the logs record no developer response to individual comments, the assignment of the *false positive* and *context-blind* examples is interpretive, based on the mismatch between the comment and the stated purpose of the PR.

7. Discussion

This section interprets the findings reported in Section 6 from three readings: self-reported acceptance of Gandalf is high, but perceived usefulness concentrates at the PR level; the dominant use posture is *calibrated trust*, with systematic maintenance of human validation; and the architectural role of the agent is limited by the absence of context beyond the diff.

7.1. Interpretation by research question

7.1.1. RQ1: Perceived ease of use, usefulness, and behavioral intention

The joint reading of the PEOU items, PU, and BI (Section 6.2) characterizes a pattern of high self-reported acceptance sustained by non-uniform perceived usefulness. Behavioral intention is the highest construct mean of the instrument and also the most internally consistent construct, while perceived ease concentrates agreement on the items that touch the agent's operational cycle, that is, learning in the review flow (q3_1) and CI/CD integration (q3_4), and weakens on the items that measure effort during the review itself (q3_2 and q3_3).

This internal contrast is consistent with the low internal consistency of PEOU, whose items capture partially distinct subdimensions and are read at the item level rather than as a reliable aggregate (Section 6.2). Read this way, the agent is perceived as easy to learn and to operate, yet it does not eliminate cognitive effort during review, a dimension in which suggestions need to be interpreted and validated. The reading is consistent with the original TAM structure (Davis, 1989), in which PEOU and PU operate partially independently on the intention to use. The persona enters this equation with a cost that the design justification does not anticipate: while the persona is intended to reduce the social friction of automated feedback (Section 3), one respondent (R3) reported that its embellishment makes the recommendations tedious to read (Section 6.4), indicating that the presumed friction reduction is not perceived uniformly and, for some readers, adds effort to the interpretation of suggestions. A configuration parameter to reduce or remove the persona's embellishment in the review text is a direct design implication of this finding, left as future evolution of the tool.

The internal heterogeneity of PU is more informative for the study design. Agreement concentrates on the items that capture perceived immediate effects on the PR, that is, detection of issues missed by human review (q4_1) and improvement of review quality (q4_2), and falls precisely on the items that touch more diffuse effects on the codebase, with emphasis on the improvement of architectural quality (q4_3, the lowest-agreement item within PU). The thematic analysis of Q8.1 corroborates this pattern: *unseen-issue-detection* is the most frequent code, and the other majority codes (*code-standardization*, *cycle-acceleration*, and *team-collaboration-support*) also describe perceived benefits local to the PR. A respondent from the customer service area (R12) condenses the dominant pattern: *"The machine never gets tired of reading code, so it doesn't let small errors and inconsistencies slip through, things that human analyses miss due to fatigue, especially on large PRs"*.

The reading, therefore, is that perceived usefulness is high in what may be called a *diff-level safety net*: a low-friction cognitive support perceived as reducing local omissions, without becoming a reasoning partner over the codebase as a whole. Modern PR-based review already privileges work on incremental changes under time pressure (Badampudi et al., 2023), and Gandalf is leveraged predominantly in this register.

7.1.2. RQ2: Trust and calibrated reliance

The Trust construct, restricted to the declared-trust items, is consistently high in both technical correctness (q5_2) and general trust (q5_1), and the separate measure of validation behavior is equally high (Section 6.3), indicating that few respondents would dispense with manual validation. Reporting the two measures separately follows from their theoretical status: declared trust and validation behavior are complementary postures, not opposite poles of a single construct, and folding the reverse-worded validation item into Trust would average together two signals that the notion of calibrated trust treats as jointly informative. In other words, trust and validation coexist in the sample: participants trust and validate.

This coexistence corresponds to the theoretical object traditionally designated as *calibrated trust* or *appropriate reliance* (Lee and See, 2004): trust is proportional to the perceived competence of the agent on each task class, rather than unconditional or absent.

Table 8 makes this pattern explicit by operationalizing the cross-tabulation between Q5.1 (declared trust) and Q5.3 (need for manual validation). The *calibrated trust* quadrant is modal and the *disengagement* quadrant (the most concerning posture for tool acceptance) is empty. The *blind trust* quadrant concentrates five respondents and deserves attention: it represents a posture in which trust is not accompanied by validation, configuring a risk of *automation bias*.

By symmetry, the *defensive distrust* quadrant corresponds to respondents who validate more than they trust, and the qualitative codes from Q8.2 explain this posture: *context-blindness* is the dominant code, with *business-rules-blindness* and *architectural-blindness* elaborating more specific categories of the same boundary. A respondent from the data engineering area (R8) illustrates how this boundary manifests in practice: *"Code reviews that depend on a broader task context. For example, modifying parts of the code that communicate with an old project or with larger structures that aren't ready yet and it flags them as wrong"*. The coherence between quantitative quadrant and qualitative codes is the central finding of RQ2: trust is contextual, calibrated by task class, and human validation operates as a stable gatekeeper. This pattern is compatible with the literature that extends TAM with trust in AI, in which the integration of Trust into the model is treated as a condition (not a substitute) for the intention to use Gefen et al. (2003), Stevens and Stetson (2023) and Alshammari and Al-Mamary (2025).

7.1.3. RQ3: Architectural and organizational implications

ArchImpact is the lowest construct mean in the sample (Section 6.4), and the item-by-item detail reveals relevant internal heterogeneity. The item of declared influence on architectural decisions (q7_1) and the item of encouragement of architectural standardization (q7_2) are, among the 19 Likert items, those with the lowest aggregate agreement of the instrument, while support for team collaboration (q7_3) and reduction of technical debt over time (q7_4) preserve respectable agreement. The perceived architectural impact, therefore, emerges *indirectly* (via consistency, standardization, and accumulation of feedback over time), not as an explicit architectural intervention by the agent on design decisions. Modern review is a natural locus where architectural decisions manifest and are renegotiated (Badampudi et al., 2023), but the actual exercise of that role by Gandalf remains embryonic in the perception of the respondents.

The thematic analysis elaborates the reading. In Q8.4 (required improvements), *repo-wide-context* is the most frequent code, with *language-domain-config*, *human-feedback-loop*, and *learning-from-history* next (Section 6.4); the code *severity-tiers* was not assigned to any respondent. Three of these codes map directly to three capabilities structurally absent in the agent's current design, declared in Section 3.6: the agent receives only the diff (context limitation), has no access to memory across PRs (lack of memory), and the prompt, although configurable per language and domain, has not been calibrated per team in the documented configurations. A respondent from the customer service area (R4) summarizes the request for additional context: *"Per-project specialization. Gandalf doesn't have enough context for code review; it needs to 'understand' what is being analyzed — seeing the 'git diff' is not enough. It must 'know' what each layer of the project does, 'know' the 'utils' and 'oops' packages, have basic 'understanding' of business rules and access to the PR description and the Jira task to 'understand' what it is analyzing"*.

In Q8.5, the use in critical projects is mostly conditional: a large majority of respondents position themselves in supervised models of use, split between *would-use-as-first-pass* and *would-use-with-double-review* (Section 6.4). The set sustains a critical reading: the perceived architectural impact reflects what the agent's current design allows and, in the perception of the users themselves, that design is recognizably insufficient for a deeper architectural role.

7.2. Comparison with related work

The first convergence with the literature is the integration into the PR flow and the sustainment of adoption in industrial contexts. Large-scale reports document continued adoption and relevant rates of comments that trigger code changes (Sun et al., 2025; Cihan et al., 2025; Ramesh et al., 2025; Tantithamthavorn et al., 2026). The present study is consistent with this adoption signal by a different route: high behavioral intention and the predominance of *would-use-as-first-pass* and *would-use-with-double-review* in Q8.5 indicate a self-reported disposition toward continued use, without measuring cycle time, comment resolution rate, or comment precision. The methodological contribution of the study is, therefore, complementary: it offers self-reported acceptance evidence via TAM extended with Trust, which can be triangulated with the process and adoption metrics that those works report.

The second convergence, more revealing, occurs along the axis of context depth. Recent technical literature invests increasing effort in expanding the context available to the LLM, whether by retrieval of relevant exemplars (Zhang et al., 2025), by benchmarks oriented to full PR context (Zhang et al., 2026; Zeng et al., 2025), or by curating datasets that bring generated comments closer to useful comments (Ben Sghaier et al., 2025). The user demand recorded in this study, with *context-blindness* leading Q8.2 and *repo-wide-context* leading Q8.4, is the perceptual counterpart of that technical direction: respondents ask, in free text, exactly for the capabilities that those works invest in building. The confluence reinforces the hypothesis that the context barrier is not only a technical limit, but an operational point of friction recognized by practitioners themselves.

In a third dimension, there is observable divergence. On PR cycle time, the literature is heterogeneous: (Tantithamthavorn et al., 2026) reports a 30.8% reduction in the evaluated scenario, while Cihan et al. (2025) reports an increase in closure duration in part of the sample. The perceptual evidence of this study, in which cycle acceleration receives only moderate item-level agreement and minority mention in Q8.1 (Section 6.2), is cautious and does not settle the point, but is compatible with the heterogeneous picture: acceleration is perceived by a minority, and the fact that *calibrated trust* is the modal quadrant, with maintenance of human validation, is consistent with scenarios in which adoption of the agent does not reduce, and may even lengthen, the time until merge.

The fourth dimension is methodological. Trust, although recognized as a central condition for AI acceptance (Gefen et al., 2003; Stevens and Stetson, 2023; Alshammari and Al-Mamary, 2025), is rarely operationalized with a validated scale in industrial studies of automated code review; most works compare models by output metrics and, at most, record trust as a qualitative anecdote. This study treats Trust

as a dependent variable measured by construct (two declared-trust items, Section 6.3), with an explicit cross-tabulation between declared trust and a separate validation-behavior measure. The central finding (*calibrated trust* as the dominant pattern and *disengagement* empty) is a methodological contribution to the debate on adoption of LLMs in code review, and not just a replication of adoption.

In parallel, the literature on cognitive load in review indicates that explicit review strategies modulate the way reviewers process changes (Gonçalves et al., 2022); the supervised use reported by the participants is consistent with this reading, since it positions Gandalf as cognitive support within a strategy of human review, not as a substitute for the reviewer.

Finally, the structural limitations reported in different studies (false positives, dependence on context, domain sensitivity, and the need for trust calibration Cihan et al., 2025; Sun et al., 2025; Zeng et al., 2025; Gonçalves et al., 2022) appear replicated in the present study from the user side, in free text. In particular, *business-rules-blindness*, *architectural-blindness*, and scenarios of *security-sensitive* and *complex-business-logic* translate, in practitioner vocabulary, the limits already recognized by the technical contributions. This mapping suggests, as a proposition to be tested in other settings, that the limits observed are not particular to Gandalf, but structural to the class of diff-only agents.

7.3. Implications for AI for software architecting

PR-based code review is, in line with Badampudi et al. (2023), a locus where architectural decisions manifest and are renegotiated, which makes the adoption of generative agents in this flow a legitimate topic in the scope of the special issue. The reported evidence, however, qualifies the expectation: the perceived architectural impact concentrates on consistency (q7_3 and q7_4), not on decision (q7_1 and q7_2), and the barrier to a more active architectural role is structural and tied to context. This diagnosis converges with the recent reading that the application of GenAI in architecture still faces gaps in precision, domain specialization, and the absence of rigorous evaluation frameworks (Esposito et al., 2026). In the terms of the levels distinguished in Section 2.6, the agent operates at the level of local code-quality feedback, with occasional incursions into design-level feedback; the codes *business-rules-blindness* and *architectural-blindness* in Q8.2 mark precisely the levels it does not reach. The three capabilities demanded in Q8.4 (repo-wide context, 10/17; language and domain specialization, 6/17; and learning from history, 2/17) correspond to three design tiers derived from participants' perceptions and from the agent's design constraints (Section 3.6). They are offered here as design implications for future work, not as validated contributions: capabilities that a review agent aligned with AI for Software Architecting would need in order to go beyond the diff level.

The first tier is the *context layer*: the capability to incorporate information beyond the diff, covering neighboring files, the PR description, the task description in management systems (R4 mentions Jira), and architectural documentation when available. Recent works that combine retrieval-augmented exemplars (Zhang et al., 2025) or that evaluate approaches in the full-PR context (Zhang et al., 2026; Zeng et al., 2025) indicate concrete directions for this tier.

The second is the *memory layer*: persistence across PRs, which allows the agent to recognize recurring organizational patterns, detect architectural drift over time, and calibrate suggestions from the history of acceptance or rejection of previous comments; LLM-driven curation of review data offers one of the primitives for this direction (Ben Sghaier et al., 2025).

The third is *tiered analysis*: explicit separation between local analysis, oriented to the diff and predominant today, and architectural analysis, oriented to the repository and to history. Although the code *severity-tiers* was not assigned to any respondent, the codes *would-use-as-first-pass* and *would-use-with-double-review* suggest that the participants already mentally operate with usage tiers; making these tiers explicit in the agent's design would align actual behavior and product affordances. Unlike the first two tiers, this third tier is an inference by the authors from these usage postures, not a demand voiced by the respondents.

The reading also has a critical side. The study evidences that high perceived local usefulness and high self-reported acceptance can coexist with low perceived architectural capability, which creates a specific risk for the AI for Software Architecting literature: taking adoption and satisfaction metrics at the PR level as evidence of architectural capability. The perceptions gathered here point to the opposite: perceived local usefulness and perceived architectural impact are empirically separable, and the findings suggest that expanding the latter requires investment in the tiers of context, memory, and tiered analysis, and not only refinements of prompting on the diff. Sustained over time, this asymmetry between local validation and architectural invariants may raise the risk of *architectural drift normalization*: the agent reinforces consistency at the diff level while no part of the workflow represents architectural-level constraints, leaving long-running systems exposed to incremental erosion that no individual PR review surfaces.

7.4. Practical implications

The findings sustain distinct recommendations according to the adoption profile. For builders of LLM-based review agents, the priority suggested by the respondents' perceptions is the context layer: the most frequently signaled barrier is *context-blindness* and the most demanded improvement is *repo-wide-context* (Section 6.4). User feedback as a continuous calibration signal (*human-feedback-loop*) appears as a second priority and memory persistence across PRs (*learning-from-history*) as a third vector.

For tech leads and engineering managers, the most robust pattern observed is the use of the agent as a *first-pass before human review* on critical projects (Section 6.4), rather than as a substitute for the human reviewer. The fraction of respondents in *blind trust* represents an organizational risk (trust without validation) and demands active monitoring, especially in teams with less experienced profiles; one respondent (R4) makes the risk explicit: *"[...] a contextless AI [...] may lead less-experienced developers into mistakes"*.

The blind-trust fraction also motivates concrete organizational safeguards against automation bias (Lee and See, 2004): keeping human review mandatory rather than optional in the presence of the agent; attaching severity levels to the agent's remarks, so that reviewers can calibrate their scrutiny; exposing confidence indicators on suggestions, signaling when the model operates outside its reliable range; training reviewers to treat the agent as a fallible first pass; and offering a lightweight mechanism to flag incorrect suggestions, which feeds calibration and gives reviewers an active role in correcting the agent. These measures turn the posture already dominant in the sample, calibrated trust, into an explicit organizational policy rather than an individual habit. In the deployment studied, the flagging mechanism is future work: no precise reporting channel exists today, and the corrections of incorrect suggestions made to the agent so far have derived from informal reports by the developers and reviewers themselves (Section 6.5).

For individual developers, the reading is that the agent is more useful as cognitive support for local errors and standardization (the dimensions in which perceived usefulness is highest) than as an arbiter of architectural decisions; the maintenance of manual validation reported by the majority is not an adoption failure, but the posture coherent with the agent's current design, and it is congruent with the literature on explicit review strategies and cognitive load (Gonçalves et al., 2022).

7.5. Threats to validity

Following the case study framework in software engineering (Runeson and Höst, 2009), threats are organized along construct, internal, external, and reliability axes, with an additional note on conflict of interest.

Construct validity: the instrument is anchored in TAM (Davis, 1989) with extension by trust in AI empirically validated in other contexts (Gefen et al., 2003; Stevens and Stetson, 2023; Alshammari and Al-Mamary, 2025), adapted to a code review agent. Internal consistency is high for PU, BI, and ArchImpact ($\alpha \geq 0.876$) and for the two-item Trust construct (inter-item $r = 0.764$; Spearman–Brown 0.866), and lower for PEOU ($\alpha = 0.586$), which is therefore read at the item level (Section 7.1.1).

Internal validity: the study is observational and focuses on self-reported perceptions; objective process measures (PR cycle time, defect rate, review coverage) are out of scope (Section 1), limiting causal inference. A possible novelty effect on the 2 respondents in the 1–3-month range is mitigated by the predominance of exposure greater than 6 months (11/17).

External validity: the sample is restricted to a single Brazilian telecommunications company (Brisanet), with stack predominantly in Go, Perl, Ruby, Flutter/Dart, React, Angular, and Next.js, and *system_domain* concentrated in customer service (8/17). Subgroup breakdowns present small n (Scrum Master $n = 1$; 1–3-month exposure $n = 2$; 3–6-month exposure $n = 4$; Tech Lead/Architect $n = 4$), so generalization requires replication in other organizations. Transferability of the findings is analytical rather than statistical: the patterns reported here are candidate propositions to be examined in other settings, not population estimates. Non-response and self-selection add to this limit: all respondents are existing users of the agent, and those who chose to answer may hold systematically more engaged views than those who did not, so the reported acceptance should be read as the perception of responding users rather than of every developer exposed to the agent. The response rate reported in Section 5.3 bounds, but does not remove, this risk.

Reliability: thematic coding of Q8.1–Q8.5 used a code book and PT→EN translations defined a priori, and all cells were independently re-coded by two additional coders blind to the original assignments, with agreement coefficients and the divergence-resolution procedure reported in Section 5.6.2. Agreement is not uniform across codes: rare codes (assigned to only a few respondents) show lower per-code coefficients, and the divergences concentrated in Q8.3 and Q8.5, where the original coding was more inferential than the blind recordings. The final assignments remain reported in Table 9 and Fig. 5 for inspection and re-coding.

Conflict of interest: Gandalf is developed at Brisanet, the same organization that hosted the study; authors Adriano Lima Cândido and Iury Lima Nogueira are employed by Brisanet, with Iury Lima Nogueira directly involved in the agent's development and deployment, creating potential for both researcher bias and organizational bias in the framing of findings. Concrete safeguards structured the data flow. The raw responses were accessed only by the first author and by the two co-authors with no ties to Brisanet; no participant was identifiable by anyone involved, including managers and the agent's developer, since the form collected no direct identification, the system-name question (Q2.1) was discarded at load time, and system purpose was reduced to the controlled vocabulary *system_domain*. These same two co-authors validated and audited the analysis and the interpretation of the results. Recruitment followed the procedure described in Section 5.3, with no involvement of managers, and participation had no bearing on performance evaluation. Social desirability remains a threat that these procedures reduce but cannot eliminate; the pattern of results, however, argues against uniform inflation: Trust and ArchImpact are among the lowest construct means, q7_1 is the lowest-agreement item of the instrument (41.2%), *blind trust* is retained and flagged as a risk, and the negative assessment of the persona (R3) is preserved and discussed. The lead author conducted data analysis and manuscript drafting in academic separation from the agent's development team. No data of direct identification of respondents is presented.

8. Conclusion and future work

This paper investigated how developers and architects perceive the use of Gandalf, a generative code review agent that operates on GitHub PRs from the diff and configurable language and domain-specific criteria, at Brisanet, a large Brazilian telecommunications company. The exploratory study combined a structured questionnaire, anchored in TAM extended with the trust-in-AI dimension, with thematic analysis of open-ended questions, and gathered 17 respondents who use the agent in projects spanning multiple stacks.

8.1. Synthesis of findings by research question

RQ1 shows high self-reported acceptance with perceived usefulness concentrated at the PR level (*unseen-issue-detection* leading Q8.1), thinning at the architectural level (q4_3 the lowest-agreement item within PU). RQ2 shows declared trust coexisting with manual validation, with *calibrated trust* as the modal quadrant and *disengagement* empty; *context-blindness* and *complex-business-logic* delimit the distrust zones, and *blind trust* deserves attention as a risk of *automation bias*. RQ3 shows the architectural impact perceived as indirect, emerging via consistency, with *repo-wide-context* leading the demand for improvements and supervised use modal in critical projects (Section 6).

8.2. Contributions

The contributions of this paper, aligned with Section 1, are three. C1 is the industrial study of practitioners' perceptions of a diff-only LLM review agent in production, paired with a documented technical description of the agent (integration with GitHub via App, orchestration via n8n, prompting strategy with criteria configurable per language and domain, standardized output format) and of the structural limitations that constrain this class of agents. C2 is the empirical evidence of *calibrated trust* as the dominant reliance posture, measured through an explicit cross-tabulation between declared trust and the separate validation-behavior measure. C3 is the set of practical requirements for evolving from diff-level review toward architecture-aware review, elicited from participants' perceptions and from the agent's design constraints and framed as design implications for future work within AI for Software Architecting (Section 7.3).

8.3. Limitations

The limits of the study are organized in four axes and discussed in detail in Section 7.5. Construct validity is supported by high internal consistency in PU, BI, ArchImpact, and Trust, while the lower consistency of PEOU leads to an item-level reading, interpreted as heterogeneity across subdimensions. Internal validity is constrained by the observational character and the focus on self-reported perceptions; objective process measures remain out of scope. In external validity, the sample (N = 17) is restricted to Brisanet, with concentration in customer service and prolonged use; transferability is analytical rather than statistical (Section 7.5). Reliability is mitigated by complete reporting of thematic codes. The declared conflict of interest reflects Gandalf's development at Brisanet.

8.4. Future work

Four research directions follow from the findings. The first is a longitudinal study that combines self-reported perceptions with objective process measures, in particular PR cycle time, agent comment resolution rate, and review coverage, allowing the perceptual evidence reported here to be triangulated with quantitative signals comparable to those of Sun et al. (2025), Cihan et al. (2025) and Tantithamthavorn et al. (2026). For the architectural dimension in particular, this triangulation should track architecture-sensitive metrics before and after adoption (dependency structure, modularity and architecture-smell trends, drift indicators) and follow the fate of agent comments that touch architectural concerns, that is, their acceptance rate and the code changes they trigger. The second is the replication in organizations from other sectors, with other stacks and team profiles, in order to test to what extent the observed pattern of *calibrated trust* and the centrality of *context-blindness* hold beyond the telecommunications context with strong presence of customer service.

The third direction is the experimental design of the three architectural extensions discussed in Section 7.3 (context layer, memory layer, and tiered analysis), each admitting isolated evaluation by perceived-usefulness delta, suggestion acceptance rate, and impact on items q7_1 and q7_2, today the lowest-agreement items in the instrument. The fourth direction is the study of the *blind trust* quadrant (Section 6) as an organizational risk condition, with instruments aimed at active calibration of trust and process safeguards that contain automation bias in teams with less experienced profiles.

In summary, in the case studied, Gandalf is perceived as a useful supplementary assistant at the PR level, used under a predominant posture of *calibrated trust*, with an architectural role that the users still perceive as indirect and limited. Evolving from *diff-level safety net* toward architecture-aware review tooling, the findings suggest, requires repository-wide context, memory across PRs, knowledge of the system's history, and the separation between local and architectural analysis, and not only refinements in prompting; the present study offers these requirements as an agenda, to be validated by future work, for the literature on AI for Software Architecting.

CRediT authorship contribution statement

Adriano Lima Cândido: Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Fernando Antonio Mota Trinta:** Writing – review & editing, Validation, Supervision, Methodology, Conceptualization. **Lincoln Souza Rocha:** Writing – review & editing, Validation, Supervision, Methodology, Conceptualization. **Iury Lima Nogueira:** Writing – review & editing, Software, Resources, Investigation.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used ChatGPT (OpenAI) and Gemini (Google) to support the development of the analysis scripts that generate the figures (F1–F7) and tables (T1–T6) reported in the manuscript, and to assist in the organization of the public replication package that accompanies the study. These tools were also used occasionally for English-language polishing of selected passages of the manuscript text. After using these tools, the authors reviewed and edited all content as needed and take full responsibility for the content of the publication.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Iury Lima Nogueira reports administrative support was provided by Brisanet Telecomunicações S.A. Adriano Lima Candido reports administrative support was provided by Brisanet Telecomunicações S.A. Co-authors Adriano Lima Cândido (A.L.C.) and Iury Lima Nogueira (I.L.N.) are employed by Brisanet Telecomunicações S.A., the organization that developed the Gandalf agent investigated in this study and that hosted the empirical evaluation. I.L.N. was directly involved in the agent's development and deployment, while A.L.C. led the research design, data analysis, and manuscript drafting in academic separation from the agent's development team. A.L.C., I.L.N. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors thank Francisco Anderson de Almada Gomes for independently re-coding the full set of open-ended responses as part of the reliability analysis (Section 5.6.2). The authors also thank Jackson Soares Martins for retrieving and organizing the GitHub records of the agent's deployment, including the review logs and invocation events that made the operational analysis reported in Section 6.5 possible.

Appendix. Survey instrument

This appendix reproduces the full data collection instrument. The questionnaire was administered in Portuguese through Google Forms; the items below were translated into English by the authors. The first question of the online form recorded informed consent (Section 5.7). Items in Blocks 3–7 use the same five-point Likert scale, stated here once: *Strongly disagree*; *Partially disagree*; *Neither agree nor disagree*; *Partially agree*; *Strongly agree*. Block 8 contains open-ended questions.

Block 1 — participant profile

Q1.1 — State your level of professional experience in IT projects. *(<2 years; 2–5 years; 6–10 years; >10 years)*
Q1.2 — Indicate your predominant role in the project(s) you have worked on. *(Developer; Senior Developer; Tech Lead/Architect; Other)*
Q1.3 — Indicate your prior familiarity with automated code review tools. *(None; Low; Moderate; High; Very high)*
Q1.4 — State your time of use of/contact with the Gandalf tool. *(<1 month; 1–3 months; 3–6 months; >6 months)*

Block 2 — project context

Instruction given to respondents: if you have used Gandalf in more than one project, answer the following questions considering exclusively the system with which you had the most interaction with Gandalf.

Q2.1 — State the name of the system/project considered. *(Discarded at data load time for anonymization; Section 5.7.)*
Q2.2 — What was your level of involvement with this system? *(Low: sporadic interaction; Medium: recurring participation; High: frequent or central activity in the project)*
Q2.3 — Briefly describe the purpose of the system (e.g., financial, customer service, sales, internal support). *(Normalized to the controlled vocabulary* `system_domain`*; Section 5.7.)*
Q2.4 — In your perception, what is the level of impact of this system for the organization? *(Low: failures do not generate relevant impact; Medium: failures generate moderate operational impact; High: failures generate significant losses or directly affect customers; Cannot assess)*

Q2.5 — How would you describe the stage of the system at the time you used Gandalf? *(Early development; Continuous evolution; Stable; Legacy, mostly maintenance; Cannot assess)*
Q2.6 — What was the approximate size of the team involved with this system when you worked on the project? *(1–5 people; 6–10 people; 11–20 people; more than 20 people; Cannot say)*

Block 3 — perceived ease of use (likert)

Q3.1 — It is easy to learn how to use Gandalf in the code review flow.
Q3.2 — The interaction with Gandalf is clear and understandable.
Q3.3 — Using Gandalf requires little additional effort during the review process.
Q3.4 — Integrating Gandalf into the development flow (e.g., CI/CD) is simple.

Block 4 — perceived usefulness (likert)

Q4.1 — Gandalf contributes to identifying problems that could go unnoticed by human reviewers.
Q4.2 — Gandalf's suggestions improve the quality of the code review.
Q4.3 — Gandalf contributes to improving the architectural quality of the code.
Q4.4 — Gandalf reduced the cognitive effort during code review.
Q4.5 — Gandalf contributes to accelerating the code review cycle.

Block 5 — trust in AI (likert)

Q5.1 — I trusted the recommendations provided by Gandalf.
Q5.2 — Gandalf's suggestions were technically correct in most cases.
Q5.3 — I felt the need to manually validate most of Gandalf's suggestions. *(Reverse-worded with respect to the declared-trust items in the original instrument; analyzed separately as a validation-behavior measure, Section 5.4.)*

Block 6 — behavioral intention (likert)

Q6.1 — I would accept the continued use of Gandalf in my workflow.
Q6.2 — I intend to keep using Gandalf whenever possible.
Q6.3 — I would recommend the use of Gandalf to other teams.

Block 7 — architectural and organizational impact (likert)

Q7.1 — Gandalf influenced relevant architectural decisions in the project.
Q7.2 — Gandalf encouraged greater architectural standardization.
Q7.3 — The use of Gandalf contributed positively to collaboration among team members.
Q7.4 — Gandalf can contribute to the reduction of technical debt over time.

Block 8 — open-ended questions

Q8.1 — What were the main perceived benefits of using Gandalf?
Q8.2 — What limitations or problems did you observe?
Q8.3 — In which situations would you not trust Gandalf for code review?
Q8.4 — What improvements would be necessary to make Gandalf more useful in industrial contexts?
Q8.5 — Would you use Gandalf on critical projects? Why?

Data availability

Data is openly available in a public replication package archived on Zenodo at https://doi.org/10.5281/zenodo.19866774, containing the analysis scripts, aggregated tables T1–T7, figures F1–F7, the Q8 thematic code book, and the data dictionary. T1–T7 and F1–F7 are the archive's internal artifact names; their correspondence to the tables and figures of this manuscript is given in Section Section 5.8. The raw questionnaire responses and the raw deployment review logs are not publicly shared due to participant confidentiality and the proprietary nature of the analyzed systems; aggregated, anonymized data may be requested from the corresponding author for replication purposes.

References

Alshammari, M.M., Al-Mamary, Y.H., 2025. User acceptance of AI-powered training: extending the technology acceptance model (TAM). Future Bus. J. 11, 239. http://dx.doi.org/10.1186/s43093-025-00665-w.

Badampudi, D., Unterkalmsteiner, M., Britto, R., 2023. Modern code reviews—survey of literature and practice. ACM Trans. Softw. Eng. Methodol. 32 (4), 107:1–107:61. http://dx.doi.org/10.1145/3585004.

Ben Sghaier, O., Weyssow, M., Sahraoui, H., 2025. Harnessing large language models for curated code reviews. In: 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories. MSR, IEEE Computer Society, Los Alamitos, CA, USA, pp. 187–198. http://dx.doi.org/10.1109/MSR66628.2025.00039.

Cihan, U., Haratian, V., İçöz, A., Gül, M.K., Devran, Ö., Bayendur, E.F., Uçar, B.M., Tüzün, E., 2025. Automated code review in practice. In: 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice. ICSE-SEIP, IEEE Computer Society, Los Alamitos, CA, USA, pp. 425–436. http://dx.doi.org/10.1109/ICSE-SEIP66354.2025.00043.

Davis, F.D., 1989. Perceived usefulness, perceived ease of use, and user acceptance of information technology. MIS Q. 13 (3), 319–340. http://dx.doi.org/10.2307/249008.

Esposito, M., Li, X., Moreschini, S., Ahmad, N., Cerny, T., Vaidhyanathan, K., Lenarduzzi, V., Taibi, D., 2026. Generative AI for software architecture: Applications, challenges, and future directions. J. Syst. Softw. 231, 112607. http://dx.doi.org/10.1016/j.jss.2025.112607.

Gefen, D., Karahanna, E., Straub, D.W., 2003. Trust and TAM in online shopping: An integrated model. MIS Q. 27 (1), 51–90. http://dx.doi.org/10.2307/30036519.

Gonçalves, P.W., Fregnan, E., Baum, T., Schneider, K., Bacchelli, A., 2022. Do explicit review strategies improve code review performance? towards understanding the role of cognitive load. Empir. Softw. Eng. 27, 99. http://dx.doi.org/10.1007/s10664-022-10123-8.

Hunt, A., Thomas, D., 2019. The Pragmatic Programmer: Your Journey to Mastery, 20th Anniversary ed., second ed. Addison-Wesley Professional.

Lee, J.D., See, K.A., 2004. Trust in automation: Designing for appropriate reliance. Hum. Factors 46 (1), 50–80. http://dx.doi.org/10.1518/hfes.46.1.50_30392.

Martin, R.C., 2008. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

Martin, R.C., 2017. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.

OWASP Foundation, 2021. OWASP top 10 for web application security risks. https://owasp.org/www-project-top-ten/, (accessed 26 April 2026).

Ramesh, S., Bose, J., Singh, H., Raghavan, A.K., Roychowdhury, S., Sridhara, G., Saini, N., Britto, R., 2025. Automated code review using large language models at ericsson: An experience report. In: 2025 IEEE International Conference on Software Maintenance and Evolution. ICSME, pp. 602–607. http://dx.doi.org/10.1109/ICSME64153.2025.00061.

Rao, N., Vasilescu, B., Holmes, R., 2025. From overload to insight: Bridging code search and code review with llms. In: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering Companion. pp. 656–660. http://dx.doi.org/10.1145/3696630.3728518.

Runeson, P., Höst, M., 2009. Guidelines for conducting and reporting case study research in software engineering. Empir. Softw. Eng. 14 (2), 131–164. http://dx.doi.org/10.1007/s10664-008-9102-8.

Stevens, A.F., Stetson, P.D., 2023. Theory of trust and acceptance of artificial intelligence technology (TrAAIT): An instrument to assess clinician trust and acceptance of artificial intelligence. J. Biomed. Inform. 148, 104550. http://dx.doi.org/10.1016/j.jbi.2023.104550.

Sun, T., Xu, J., Li, Y., Yan, Z., Zhang, G., Xie, L., Geng, L., Wang, Z., Chen, Y., Lin, Q., Duan, W., Sui, K., 2025. Bitsai-cr: Automated code review via llm in practice. In: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering. FSE Companion'25, Association for Computing Machinery, New York, NY, USA, pp. 274–285. http://dx.doi.org/10.1145/3696630.3728552.

Tantithamthavorn, K., Zou, Y., Wong, A., Gupta, M., Wang, Z., Buller, M., Jiang, R., Watson, M., Jeong, M., Chen, K., Wu, M., 2026. Rovodev code reviewer: A large-scale online evaluation of llm-based code review automation at atlassian. http://dx.doi.org/10.48550/arXiv.2601.01129, arXiv preprint arXiv:2601.01129.

Zeng, Z., Shi, R., Han, K., Li, Y., Sun, K., Wang, Y., Yu, Z., Xie, R., Ye, W., Zhang, S., 2025. Benchmarking and studying the llm-based code review. http://dx.doi.org/10.48550/arXiv.2509.01494, arXiv preprint arXiv:2509.01494.

Zhang, D., Zhang, S., Jin, Z., Luo, J., Fu, S., Nallipogu, E., 2026. Sphinx: Benchmarking and modeling for llm-driven pull request review. http://dx.doi.org/10.48550/arXiv.2601.04252, arXiv preprint arXiv:2601.04252.

Zhang, Y., Zhang, Y., Sun, Z., Jiang, Y., Liu, H., 2025. Laura: Enhancing code review generation with context-enriched retrieval-augmented llm. In: Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering. pp. 2983–2995. http://dx.doi.org/10.1109/ASE63991.2025.00245.